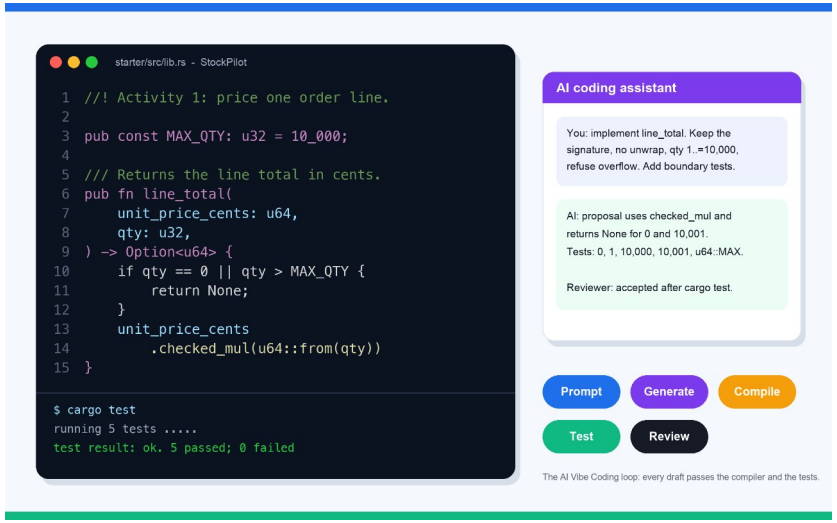
RUST

AI VIBE CODING

COURSE SLIDES · WSQ

AI Vibe Coding with Rust



Course illustration: Activity 1 code and its real cargo test result

WSQ Course Code: TGS-2023039924

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr Alfred Ang

Version 3.1 · 27 September 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

MANDATORY LEARNER ADMIN

Digital Attendance (Mandatory)

1

TRAQOM / SSG

Take the digital attendance through the Singpass app when the trainer displays the QR code.

2

AM, PM and assessment

Attendance is taken for each session and separately for the assessment.

3

75% minimum

At least 75% attendance is required for assessment eligibility and funding.

4

Your own record

Use only your own Singpass; tell the trainer if the course title or code does not match.

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



General Trainer

Complete before delivery

NAME

DESIGNATION

QUALIFICATIONS

EXPERTISE

INDUSTRY / TRAINING EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

NAMED TRAINER

About the Trainer



Dr Alfred Ang

Founder / Trainer, Tertiary Infotech Academy

COURSE
AI Vibe Coding with Rust

FOCUS
AI-assisted software development, programming and software design

TRAINING APPROACH
Specification first, vibe-coded drafts, compiler + tests as the gate

CONTACT
enquiry@tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

KNOW THE CLASS

Learner Introduction

1

Your name and role

What do you do at work today?

2

Programming background

Any language: Python, JavaScript, C, none yet?

3

AI coding tools

Which assistant have you tried: Copilot, Claude, ChatGPT?

4

Your goal

One tool or service you would like to build in Rust.

© 2026 Tertiary Infotech Academy Pte Ltd

CLASS WORKING AGREEMENT

Ground Rules

1

Your code, your call

AI drafts; you accept only what you can explain.

2

Evidence over opinion

cargo test, clippy and fmt decide "done", not the AI.

3

Privacy

Never paste passwords, keys, personal data or assessment answers into AI tools.

4

Respect

Phones on silent; ask questions early; help without giving answers.

© 2026 Tertiary Infotech Academy Pte Ltd

FIVE TOPICS / 14 HANDS-ON ACTIVITIES

Course Outline

1

Topic 1

Rust Programming Fundamentals and Software Components

2

Topic 2

AI Vibe Coding for Rust Application Development

3

Topic 3

Rust Programming Controls, Functions and Features

4

Topic 4

Testing, Debugging and Integrating Rust Components

5

Topic 5

AI-Assisted Rust Software Design and Documentation

6

Assessment

Written Assessment + Practical Performance

© 2026 Tertiary Infotech Academy Pte Ltd

9:30 AM-6:30 PM / 14 TRAINING + 2 ASSESSMENT HOURS

Lesson Plan / two days

Time	Day 1	Day 2
Morning	9:30-11:30 Topic 1 / 2h; 11:30-12:30 Topic 2 / 1h	9:30-12:30 Topic 4 / 3h
12:30-1:30 PM	Lunch / 1 hour	Lunch / 1 hour
Afternoon	1:30-3:30 Topic 2 / 2h; 3:30-6:30 Topic 3 / 3h	1:30-4:30 Topic 5 / 3h
Assessment	No assessment on Day 1	WA 4:30-5:30 PM / PP 5:30-6:30 PM

WHEN IT MATTERS Topic durations 2/3/3/3/3 hours. Day 1: 8 training hours. Day 2: 6 training hours + 2 assessment hours. Lunch excluded.

© 2026 Tertiary Infotech Academy Pte Ltd

Learning Outcomes

1

LO1

Determine basic software components using programming methodologies to meet functional specifications

2

LO2

Apply programming methodologies and tools for software creation

3

LO3

Select essential programming controls and features to meet software design requirements

4

LO4

Examine the interoperability and functionality of programming components

5

LO5

Generate programming design documentation aligned with user specifications

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE ASSESSMENT

Briefing for Assessment

1

Instruments

Written Assessment: 5 open-ended questions.
Practical Performance: 5 hands-on tasks.

2

What is assessed

Components, tools, controls/features, testing/integration and documentation.

3

Conditions

Open book, individual. AI only where a PP task explicitly allows a reviewed draft.

4

Evidence

Keep your crates, cargo output and records; upload as instructed.

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL DAY / 2 HOURS

Assessment & Funding

Written Assessment

- Day 2, 4:30-5:30 PM
- 60 minutes / 5 questions
- K1-K5 / open book

Practical Performance

- Day 2, 5:30-6:30 PM
- 60 minutes / 5 tasks
- A1-A5 / Rust crates + evidence

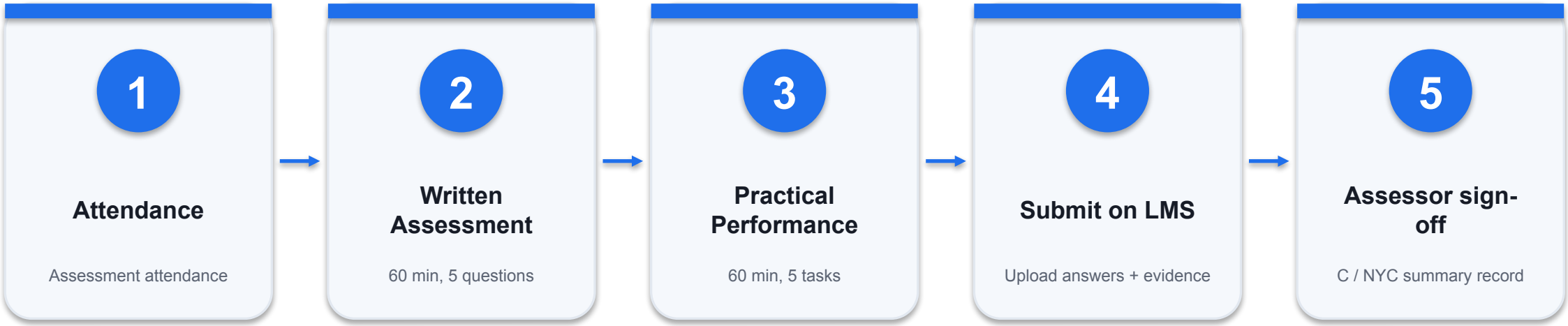
Result and funding

- All items attempted individually
- Competent (C) or NYC
- Funding requires 75% attendance + C

© 2026 Tertiary Infotech Academy Pte Ltd

OPEN BOOK / INDIVIDUAL

Assessment Flow



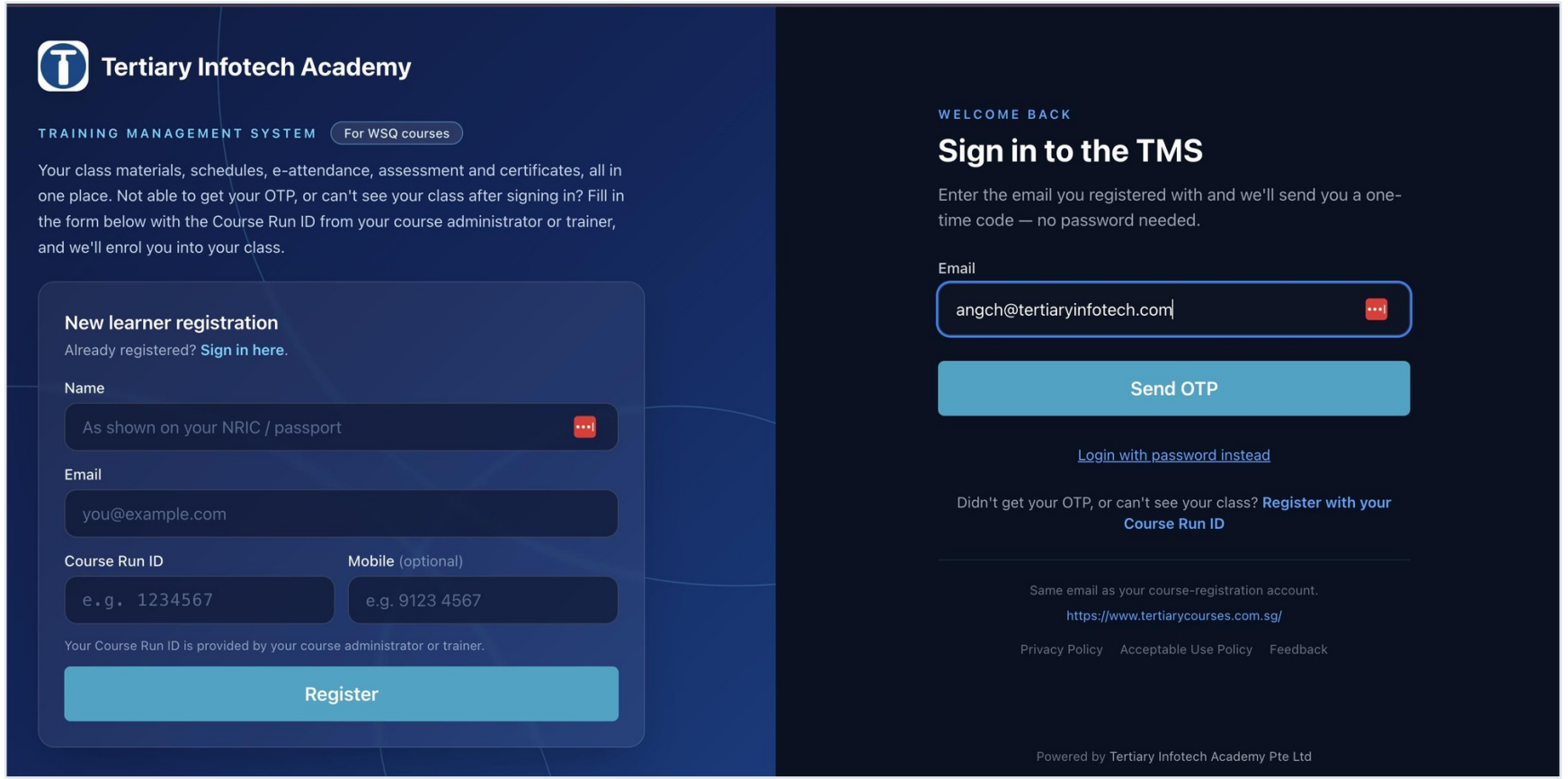
ORDER OF THE DAY

Written Assessment (K1-K5) 4:30-5:30 PM, then Practical Performance (A1-A5) 5:30-6:30 PM on Day 2. Follow the assessor's submission instructions.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

Login to LMS and Download Course Material

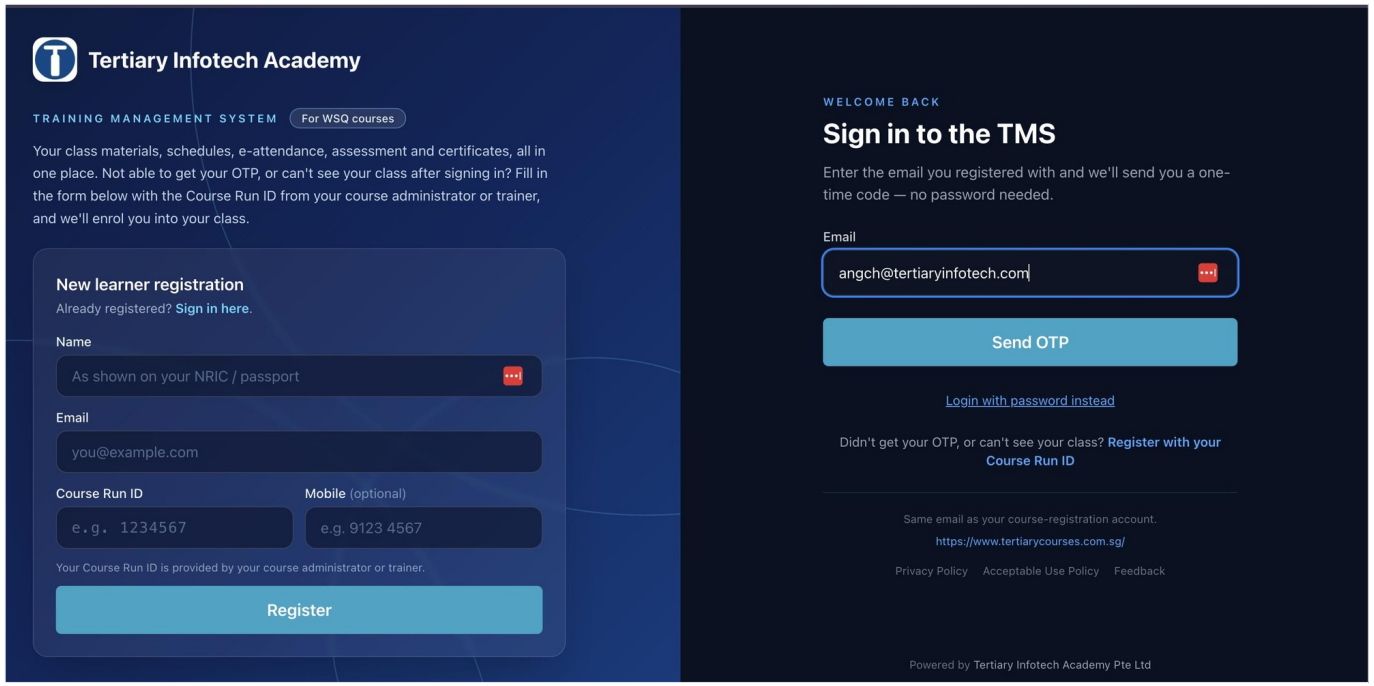


Log in at lms-tms.tertiaryinfotech.com with your registered email and the OTP sent to your inbox, then open this course to download the slides and Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNER ACCESS

Download course material from the LMS



Screenshot of the live LMS sign-in page (lms-tms.tertiaryinfotech.com), captured September 2026.

1. Sign in

Open lms-tms.tertiaryinfotech.com and sign in with your email (OTP) or password.

2. Open this course

AI Vibe Coding with Rust (TGS-2023039924).

3. Download

Learner slides PDF, Learner Guide and the Activities link.

4. Assessment

Question papers open on the assessment day; upload your answers there.

Open the LMS

© 2026 Tertiary Infotech Academy Pte Ltd

CLICKABLE LEARNER REPOSITORY

Get the Hands-On Activities

Course repository

- 14 activity folders
- starter/ + solution/ crates
- Data files and prompts

Get a working copy

- git clone, or
- Download ZIP from GitHub
- Open a folder in VS Code

Toolchain check

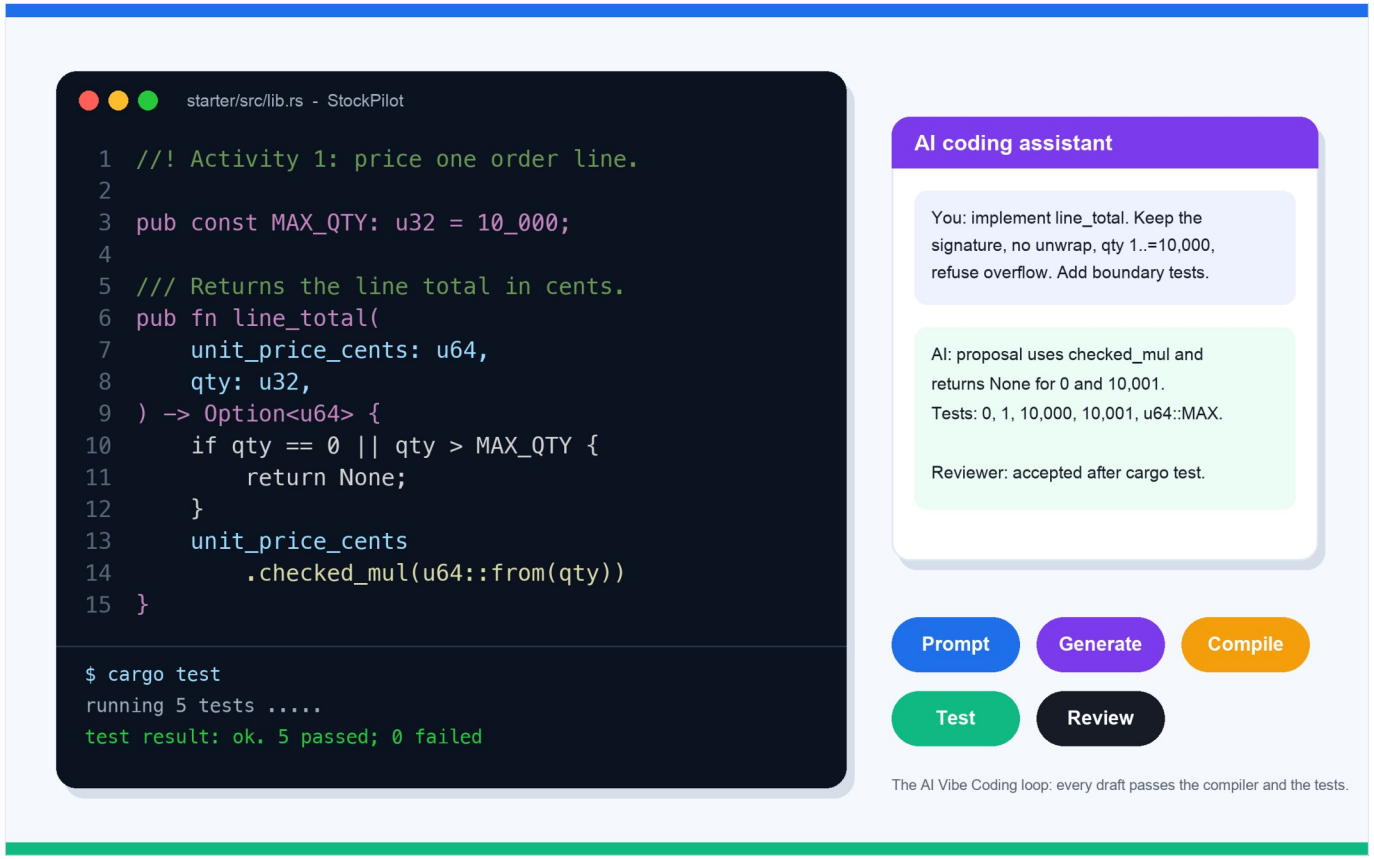
- rustc --version
- cargo --version
- cargo test in solution/

https://github.com/tertiarycourses/TGS-2023039924-AI-Vibe-Coding-with-Rust

© 2026 Tertiary Infotech Academy Pte Ltd

HOW WE WORK

Your AI Vibe Coding workbench



Course illustration; the code and test result shown are from Activity 1 of this course.

Editor

VS Code + rust-analyzer shows types and errors as you type.

AI assistant

Drafts code, tests and docs from a bounded prompt.

Cargo

Builds, tests, lints and formats: the objective gate.

You

Decide what is accepted, and record why.

© 2026 Tertiary Infotech Academy Pte Ltd

Learn Rust: recommended resources

Rust official learning hub

https://rust-lang.org/learn/

W3Schools Rust tutorial

https://www.w3schools.com/rust/

W3Schools: Get Started with Rust

https://www.w3schools.com/rust/rust_getstarted.php

Programiz: Learn Rust

https://www.programiz.com/rust

IONOS Digital Guide: Rust tutorial

https://www.ionos.com/digitalguide/websites/web-development/rust-tutorial/

JetBrains: Getting Started with Rust

https://lp.jetbrains.com/getting-started-with-rust/

OneCompiler: Rust tutorial

https://onecompiler.com/tutorials/rust

It's FOSS: Rust programming tutorial series

https://itsfoss.com/rust-tutorials/

© 2026 Tertiary Infotech Academy Pte Ltd

1

TOPIC 1 / K1/A1

Rust Programming Fundamentals and Software Components

Activities 1-3 / concepts, contracts, failure modes and evidence

© 2026 Tertiary Infotech Academy Pte Ltd

A specification becomes a component contract

Read the code, then the contract: what must always be true?

```
R1 price one order line in cents
R2 quantity must be 1..=10,000
R3 never wrap or lose money on overflow

component: fn line_total(
    unit_price_cents: u64,   // input
    qty: u32,                // input
) -> Option<u64>             // output or refusal
```

Requirement

Write the rule in business words before any code exists.

Input

Each input gets a type that already excludes invalid values (u32 is never negative).

Output

Option<u64> makes "refused" part of the signature, not a comment.

Evidence

Each requirement maps to at least one named test.

© 2026 Tertiary Infotech Academy Pte Ltd

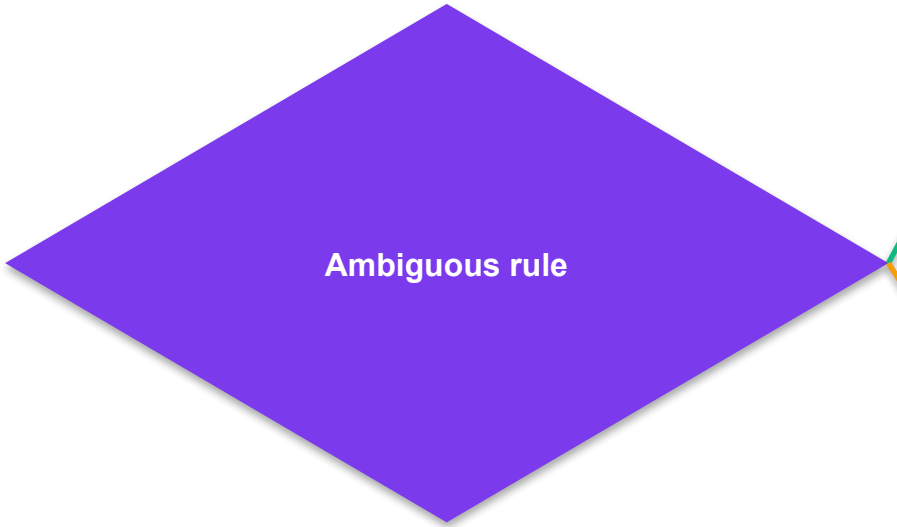
A specification becomes a component contract / contract

Contract field	Exact rule
Business rule	Totals are whole cents; quantity 1..=10,000; overflow is refused.
Rust types	u64 for money in cents, u32 for quantity, Option for refusal.
Component boundary	A pure function: no printing, no files, no global state.
Acceptance	Tests at 0, 1, 10,000, 10,001 and u64::MAX.

WHEN IT MATTERS Every requirement ID appears in a test name or a trace table row.

© 2026 Tertiary Infotech Academy Pte Ltd

A specification becomes a component contract / failure



Symptom present: fix

Ask the business owner (or the AI, then verify) before coding: is 0 an error or a no-op?

Contract holds: evidence

Every requirement ID appears in a test name or a trace table row.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

The Rust toolchain: rustup, cargo and rustc

Read the code, then the contract: what must always be true?

```
# install once (macOS/Linux; Windows uses rustup-init.exe)
curl https://sh.rustup.rs -sSf | sh
rustup component add clippy rustfmt

rustc --version      # the compiler
cargo --version      # build tool + package manager
cargo new stockpilot # create a package
cargo run            # build and run
cargo test           # build and run tests
```

rustup

Installs and updates Rust toolchains and components.

cargo

One command for new, build, run, test, fmt, clippy and doc.

rustc

The compiler; cargo calls it for you with the right flags.

stable

The course uses the stable channel; record the version as evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / EXACT INTERFACE

The Rust toolchain: rustup, cargo and rustc / contract

Contract field	Exact rule
Install	rustup from rustup.rs; Windows also needs the MSVC build tools.
Editor	Visual Studio Code with the rust-analyzer extension.
Components	clippy (lints) and rustfmt (formatting) via rustup component add.
Evidence	rustc --version output is recorded in every verification record.

WHEN IT MATTERS rustc --version and cargo --version both print a version in the VS Code terminal.

© 2026 Tertiary Infotech Academy Pte Ltd

The Rust toolchain: rustup, cargo and rustc / failure



Symptom present: fix

Open a new terminal so PATH includes ~/.cargo/bin, or run the rustup installer again.

Contract holds: evidence

rustc --version and cargo --version both print a version in the VS Code terminal.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Anatomy of a Cargo package

Read the code, then the contract: what must always be true?

```
activity01/
  Cargo.toml          # name, edition, dependencies
  Cargo.lock          # exact resolved versions
  src/
    lib.rs            # library crate: the logic
    main.rs           # binary crate: thin entry point
  tests/
    acceptance.rs     # integration tests (public API)
  target/             # build output (never commit)
```

Cargo.toml

The manifest: package metadata and dependencies.

lib.rs

Holds the reusable, testable components.

main.rs

Only parses arguments, calls the library and prints results.

tests/

Each file is compiled as a separate crate that uses the public API.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / EXACT INTERFACE

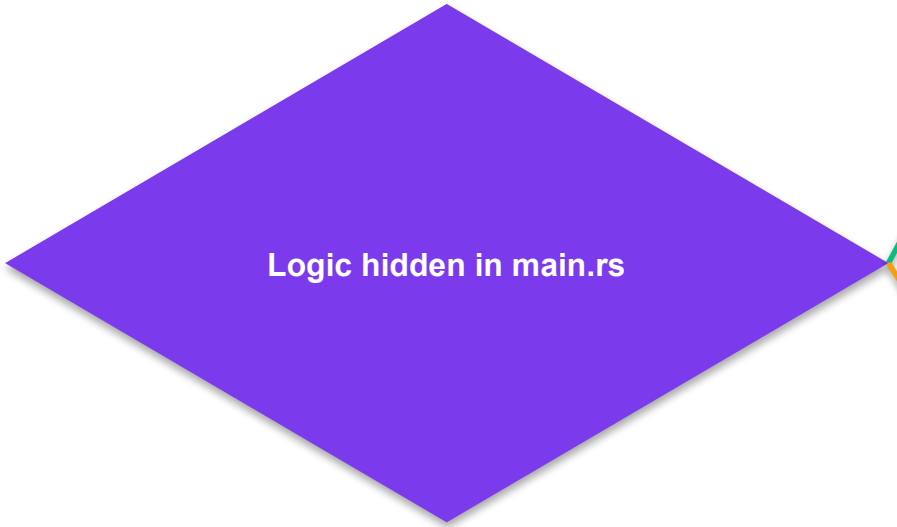
Anatomy of a Cargo package / contract

Contract field	Exact rule
Package	One Cargo.toml; can contain one library and several binaries.
Crate	A compilation unit: lib.rs or main.rs and its modules.
Edition	edition = "2021" fixes the language rules the crate uses.
Build output	target/ is generated; add it to .gitignore.

WHEN IT MATTERS cargo test builds lib, bin and tests; tests only use pub items.

© 2026 Tertiary Infotech Academy Pte Ltd

Anatomy of a Cargo package / failure



Symptom present: fix

Move it into lib.rs so integration tests can call it directly.

Contract holds: evidence

cargo test builds lib, bin and tests; tests only use pub items.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Variables are immutable by default

Read the code, then the contract: what must always be true?

```
let qty = 4;                // immutable binding
let mut stock = 10;         // mutable binding
stock -= qty;               // allowed: declared mut

const MAX_QTY: u32 = 10_000;   // compile-time constant

let input = " 42 ";
let input: u32 = input.trim().parse().unwrap_or(0);
// shadowing: same name, new type and value
```

let

A binding cannot change unless you opt in with mut.

mut

Makes every place that changes state easy to find in review.

const

Named, typed constant; UPPER_SNAKE_CASE by convention.

Shadowing

Re-using a name with let creates a new variable, often of a new type.

© 2026 Tertiary Infotech Academy Pte Ltd

Variables are immutable by default / contract

Contract field	Exact rule
Default	Immutable: accidental writes are compile errors (E0384).
Constants	Must have an explicit type; evaluated at compile time.
Naming	snake_case for variables and functions, CamelCase for types.
Review rule	Every mut should be necessary; clippy flags unused mut.

WHEN IT MATTERS The code compiles with no unused_mut warnings under clippy.

© 2026 Tertiary Infotech Academy Pte Ltd

Variables are immutable by default / failure



Symptom present: fix

Add mut only if the value truly changes; otherwise create a new binding.

Contract holds: evidence

The code compiles with no unused_mut warnings under clippy.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / FORMULA

Choose integer types deliberately

Read the code, then the contract: what must always be true?

```
let unit: u64 = 250;                // cents, never float
let qty: u32 = 4;                   // cannot be negative
let total = unit * u64::from(qty);   // widen, then multiply

u64::MAX.checked_mul(2)             // None
250u64.checked_mul(4)               // Some(1000)
3u32.checked_sub(5)                 // None
(-4i64).abs()                       // 4
u32::try_from(-1i64)                // Err(..)
```

Money

Store cents in an integer; floats cannot represent 0.10 exactly.

Widening

u64::from(qty) converts losslessly before multiplying.

checked_*

Returns None instead of overflowing.

try_from

Narrowing conversions return Result because they can fail.

© 2026 Tertiary Infotech Academy Pte Ltd

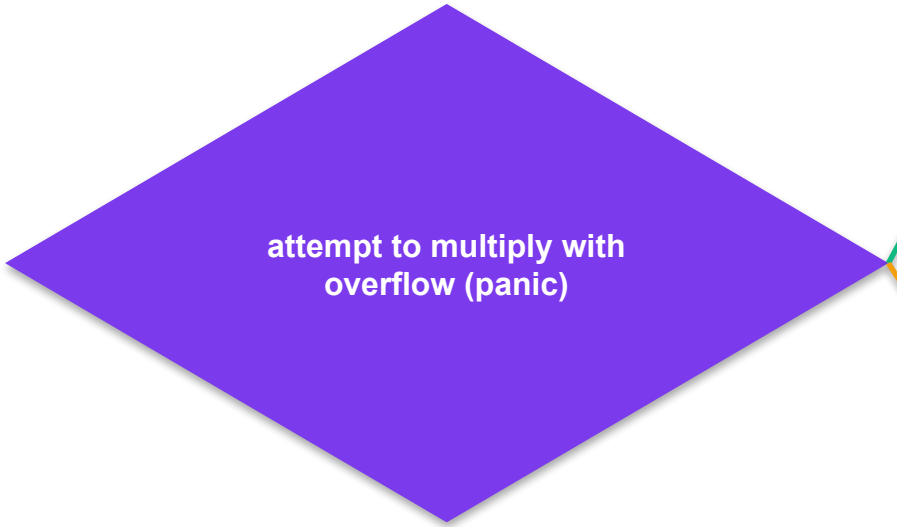
Choose integer types deliberately / contract

Contract field	Exact rule
u32	0..=4,294,967,295: quantities and counts.
u64	0..=18.4 quintillion: money in cents and large totals.
i64	Signed: adjustments that can be negative.
Overflow policy	Debug builds panic, release builds wrap: so use checked_* explicitly.

WHEN IT MATTERS Tests at u64::MAX and u32::MAX prove the overflow policy.

© 2026 Tertiary Infotech Academy Pte Ltd

Choose integer types deliberately / failure



Symptom present: fix

Replace * with checked_mul and return None or an error to the caller.

Contract holds: evidence

Tests at u64::MAX and u32::MAX prove the overflow policy.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Functions: the signature is the contract

Read the code, then the contract: what must always be true?

```
pub fn line_total(
    unit_price_cents: u64,
    qty: u32,
) -> Option<u64> {
    if qty == 0 || qty > MAX_QTY {
        return None;              // early return
    }
    unit_price_cents.checked_mul(u64::from(qty))
    // last expression, no semicolon = return value
}
```

Parameters

Every parameter has an explicit type; nothing is inferred at the boundary.

Return type

The arrow type tells callers what they must handle.

Expression body

The final expression without ; is the value returned.

pub

Only pub functions are part of the crate API.

© 2026 Tertiary Infotech Academy Pte Ltd

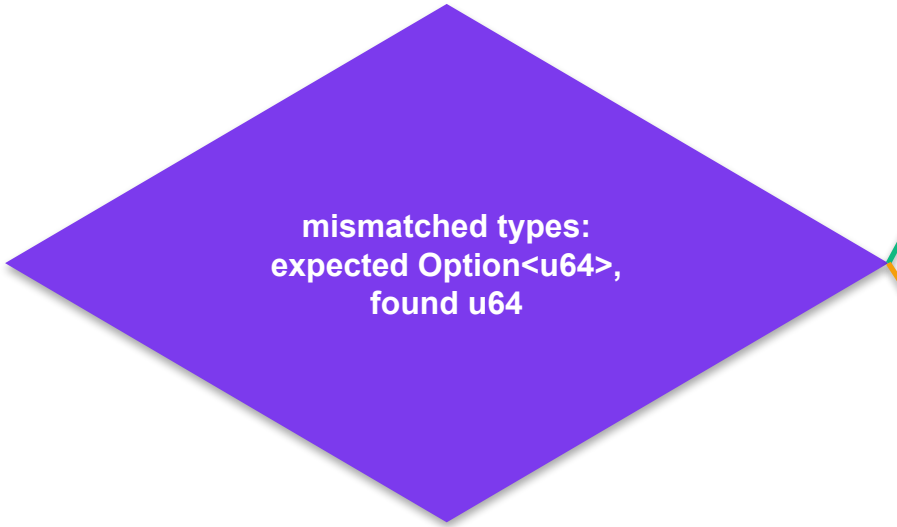
Functions: the signature is the contract / contract

Contract field	Exact rule
Inputs	unit_price_cents: u64, qty: u32.
Output	Some(total) or None, never a panic.
Side effects	None: no printing, files or globals.
Tests	Normal, zero, maximum, above maximum and overflow.

WHEN IT MATTERS Five boundary tests in Activity 1 pass.

© 2026 Tertiary Infotech Academy Pte Ltd

Functions: the signature is the contract / failure



Symptom present: fix

Wrap the success value in Some(...) or return the checked_* result directly.

Contract holds: evidence

Five boundary tests in Activity 1 pass.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / CODE

String and &str: owned text and borrowed text

Read the code, then the contract: what must always be true?

```
let owned: String = String::from("abc-1234");
let view: &str = &owned;           // borrow, no copy
let trimmed: &str = "  ABC-1234 ".trim();

fn is_valid_sku(code: &str) -> bool { /* reads */ }
fn normalise_sku(code: &str) -> Option<String> {
    let upper = code.trim().to_ascii_uppercase(); // new String
    is_valid_sku(&upper).then_some(upper)
}
```

String

Owns growable UTF-8 text on the heap.

&str

A borrowed view of text; the preferred parameter type.

Returning String

Needed only when the function creates new text.

UTF-8

Byte length and character count differ for non-ASCII text.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / EXACT INTERFACE

String and &str: owned text and borrowed text / contract

Contract field	Exact rule
Parameter	Accept &str so callers can pass literals or &String.
Return	String when new text is produced; &str when returning part of an input.
Validation	ASCII rules can use bytes safely; general text should use chars().
Tests	Include whitespace, lower case and a non-ASCII letter.

WHEN IT MATTERS Activity 2 tests pass, including "ÄBC-1234" being rejected.

© 2026 Tertiary Infotech Academy Pte Ltd

String and &str: owned text and borrowed text / failure



Symptom present: fix

Pass a borrow: &my_string, or change the parameter to &str.

Contract holds: evidence

Activity 2 tests pass, including "ÄBC-1234" being rejected.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / CODE

Ownership moves; borrowing lends

Read the code, then the contract: what must always be true?

```
let items = vec![cable, charger];

let total = total_quantity(&items);   // shared borrow
restock(&mut items[0], 3);            // exclusive borrow
let kept = in_stock(items);           // move: items is gone

// println!("{:?}", items);
// error[E0382]: borrow of moved value: `items`
```

Owner

Every value has exactly one owner; it is dropped when the owner goes out of scope.

&T

Any number of readers at once; nobody may change the value.

&mut T

Exactly one writer and no readers at the same time.

Move

Passing by value transfers ownership; the old name cannot be used.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / EXACT INTERFACE

Ownership moves; borrowing lends / contract

Contract field	Exact rule
Read only	fn f(items: &[Item]) - caller keeps ownership.
Change in place	fn f(item: &mut Item) - caller sees the change.
Consume	fn f(items: Vec<Item>) -> Vec<Item> - caller hands the value over.
Clone	An explicit copy; justify it in review because it costs memory and time.

WHEN IT MATTERS Activity 3 compiles with no clone() and all five tests pass.

© 2026 Tertiary Infotech Academy Pte Ltd

Ownership moves; borrowing lends / failure



Symptom present: fix

Borrow instead of moving (&items), or clone only if two owners are truly needed.

Contract holds: evidence

Activity 3 compiles with no clone() and all five tests pass.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / 25 MIN HANDS-ON

Turn a specification into a Rust function

ACTIVITY 1

Independent practice; full steps in the activity README and Learner Guide

StockPilot must price an order line. The specification limits quantity to 1..10,000 and forbids a silently wrapped total. You create the Cargo package, write the function and prove it with tests.

YOU'LL BUILD

labs/activity-01-line-total: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 5 passed; cargo run prints "4 x 250 cents = 1000 cents".

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 / LEARN RUST

Turn a specification into a Rust function / learn it first

samples/01_variables_and_types.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: variables, mutability, constants and integer types.
const MAX_QTY: u32 = 10_000;

fn main() {
    let unit_price_cents: u64 = 250; // immutable by default
    let mut qty: u32 = 4; // mut: this value will change
    println!("price = {unit_price_cents} cents, qty = {qty}");
    qty += 1;
    println!("after adding one: qty = {qty} (max {MAX_QTY})");
    let total = unit_price_cents * u64::from(qty);
    println!(
        "total = {total} cents = ${}.{:02}",
        total / 100,
        total % 100
    );
    let big: u64 = u64::MAX;
// ... full script: samples/01_variables_and_types.rs
```

Run the sample

rustc --edition 2021 01_variables_and_types.rs && ./01_variables_and_types

Explain it simply

Ask the AI to explain Cargo, variables, mutability, constants, integer types and functions that return Option.

Quiz me

5 questions, one at a time, including a predict-the-output question.

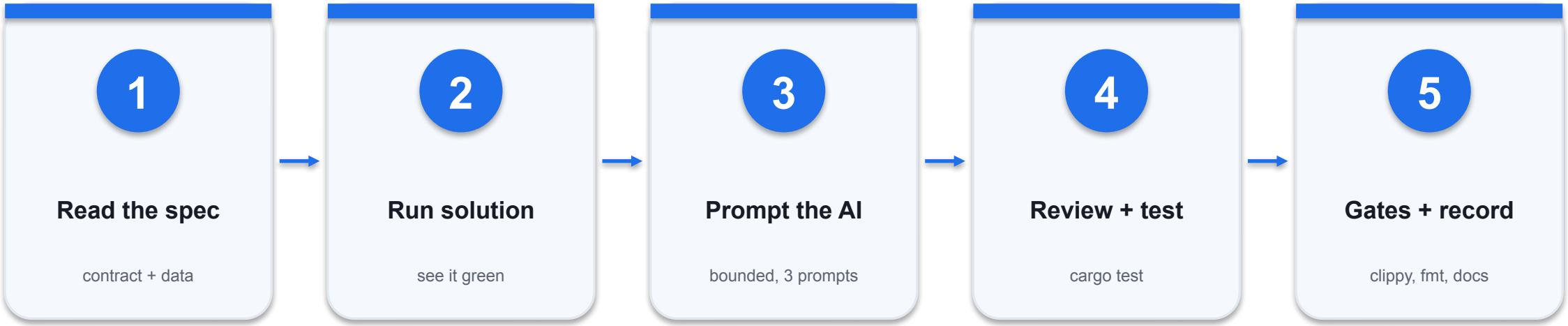
Compare with what I know

Rust vs Python / JavaScript for variables, integer types and overflow.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 / WORKFLOW

Turn a specification into a Rust function / workflow



EXACT CONTRACT

line_total(unit_price_cents: u64, qty: u32) -> Option<u64>. qty 0 or above MAX_QTY (10,000) returns None; an overflowing product returns None; otherwise Some(total cents).

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 / CODE

Turn a specification into a Rust function / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Returns the line total in cents.
///
/// Returns `None` when `qty` is outside `1..=MAX_QTY` or when the
/// multiplication would overflow `u64`.
pub fn line_total(unit_price_cents: u64, qty: u32) -> Option<u64> {
    if qty == 0 || qty > MAX_QTY {
        return None;
    }
    unit_price_cents.checked_mul(u64::from(qty))
}
```

Specification

Quantity range and overflow policy are decided before any code is written.

Types

u64 cents avoid floating-point rounding; u32 quantity cannot be negative.

Option

None is an explicit, typed rejection that the caller must handle.

checked_mul

Overflow becomes None instead of wrapping or panicking.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Turn a specification into a Rust function / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: line_total(unit_price_cents: u64, qty: u32) -> Option<u64>. qty 0 or above MAX_QTY (10,000) returns None; an overflowing product returns None; otherwise Some(total cents). Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Turn a specification into a Rust function" in starter/src so that the supplied tests pass. Keep the u64/u32 types and the Option return; use checked arithmetic only. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 / EVIDENCE

Turn a specification into a Rust function / acceptance

✅ Expected result

cargo test reports 5 passed; cargo run prints "4 x 250 cents = 1000 cents". Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 / 25 MIN HANDS-ON

Validate a product code with strings

ACTIVITY 2

Independent practice; full steps in the activity README and Learner Guide

Product codes arrive from scanners and spreadsheets with stray spaces and lower-case letters. StockPilot accepts only the SKU format AAA-9999 and normalises valid input to upper case.

YOU'LL BUILD

labs/activity-02-sku-validation: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 6 passed, including the non-ASCII and whitespace cases.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 2 / LEARN RUST

Validate a product code with strings / learn it first

samples/01_string_and_str.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: owned String versus borrowed &str, and bytes versus characters.
fn describe(text: &str) {
    println!(
        "{text:?}: {} bytes, {} chars",
        text.len(),
        text.chars().count()
    );
}

fn main() {
    let owned: String = String::from("  abc-1234  ");
    let trimmed: &str = owned.trim(); // borrows part of `owned`
    let upper: String = trimmed.to_ascii_uppercase(); // new owned text
    describe(&owned);
    describe(trimmed);
    describe(&upper);
// ... full script: samples/01_string_and_str.rs
```

Run the sample

rustc --edition 2021 01_string_and_str.rs && ./01_string_and_str

Explain it simply

Ask the AI to explain String versus &str, trimming, UTF-8 bytes versus chars, and validating text.

Quiz me

5 questions, one at a time, including a predict-the-output question.

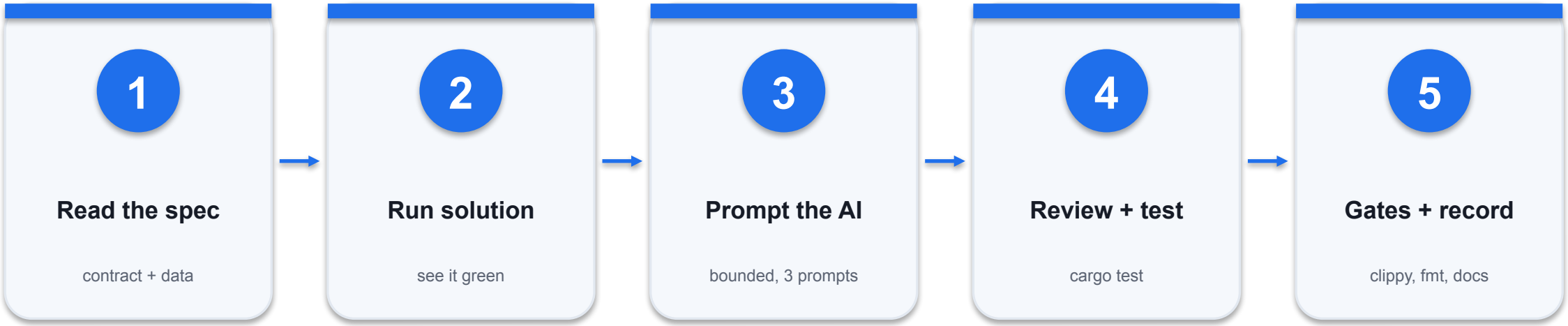
Compare with what I know

Rust vs Python / JavaScript for strings and text.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 2 / WORKFLOW

Validate a product code with strings / workflow



EXACT CONTRACT

is_valid_sku(&str) -> bool: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits. normalise_sku(&str) -> Option<String>: trimmed upper-case SKU when valid, else None.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 2 / CODE

Validate a product code with strings / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Returns true when `code`, after trimming, has the form `AAA-9999`.
pub fn is_valid_sku(code: &str) -> bool {
    let bytes = code.trim().as_bytes();
    bytes.len() == 8
        && bytes[..3].iter().all(u8::is_ascii_uppercase)
        && bytes[3] == b'-'
        && bytes[4..].iter().all(u8::is_ascii_digit)
}
```

&str

The validator borrows text; it never needs to own or copy it.

trim

Returns a slice of the same text without leading or trailing whitespace.

Bytes

The format is ASCII, so byte positions are exact character positions.

String

normalise_sku returns an owned String because it creates new text.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 2 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Validate a product code with strings / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: is_valid_sku(&str) -> bool: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits. normalise_sku(&str) -> Option<String>: trimmed upper-case SKU when valid, else None. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Validate a product code with strings" in starter/src so that the supplied tests pass. Borrow &str inputs, return an owned String only from normalise_sku, and do not use regex crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 2 / EVIDENCE

Validate a product code with strings / acceptance

✅ Expected result

cargo test reports 6 passed, including the non-ASCII and whitespace cases. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K1/A1 K3/A3 / 25 MIN HANDS-ON

Share inventory records with ownership and borrowing

ACTIVITY 3

Independent practice; full steps in the activity README and Learner Guide

Three StockPilot functions handle the same item list differently: one only reads it, one changes one item and one consumes the list. You choose the parameter types that express those intentions.

YOU'LL BUILD

labs/activity-03-ownership-borrowing: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 5 passed; the overflow test proves the item is unchanged.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 3 / LEARN RUST

Share inventory records with ownership and borrowing / learn it first

samples/01_move_and_clone.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: moving ownership, cloning, and Copy types.
fn consume(items: Vec<String>) -> usize {
    items.len() // `items` is dropped at the end of this function
}

fn main() {
    let items = vec![String::from("cable"), String::from("charger")];
    let copy = items.clone(); // explicit deep copy
    let count = consume(items); // ownership moves into `consume`
                                // println!("{:?}", items); // error[E0382]: borrow of
moved value
    println!("consumed {count} items; the clone still has {copy:?}");
    let n: u32 = 5;
    let m = n; // integers are Copy: both stay usable
    println!("n = {n}, m = {m}");
}
```

Run the sample

rustc --edition 2021 01_move_and_clone.rs && ./01_move_and_clone

Explain it simply

Ask the AI to explain ownership, moves, Copy types, clone, shared borrows (&) and mutable borrows (&mut).

Quiz me

5 questions, one at a time, including a predict-the-output question.

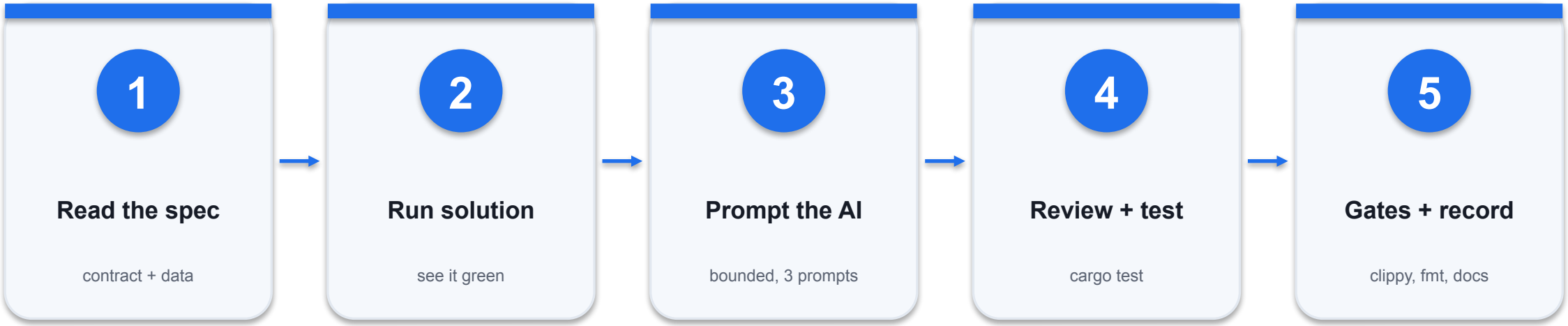
Compare with what I know

Rust vs Python / JavaScript for ownership and borrowing (memory management).

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 3 / WORKFLOW

Share inventory records with ownership and borrowing / workflow



EXACT CONTRACT

Item { sku, name, qty: u32 }. total_quantity(&[Item]) -> u64 reads; restock(&mut Item, u32) -> Option<u32> changes one item and leaves it unchanged on overflow; in_stock(Vec<Item>) -> Vec<Item> consumes the list.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 3 / CODE

Share inventory records with ownership and borrowing / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Borrows one item mutably and returns its new quantity.
///
/// On overflow the item is left unchanged and `None` is returned.
pub fn restock(item: &mut Item, amount: u32) -> Option<u32> {
    let new_qty = item.qty.checked_add(amount)?;
    item.qty = new_qty;
    Some(new_qty)
}
```

&[Item]

A shared borrow: many readers, no changes, caller keeps ownership.

&mut Item

An exclusive borrow: exactly one writer while the borrow lives.

Vec<Item>

Passing by value moves ownership; the caller can no longer use the list.

? on Option

checked_add(...)? returns None early before the item is modified.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 3 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Share inventory records with ownership and borrowing / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: Item { sku, name, qty: u32 }. total_quantity(&[Item]) -> u64 reads; restock(&mut Item, u32) -> Option<u32> changes one item and leaves it unchanged on overflow; in_stock(Vec<Item>) -> Vec<Item> consumes the list. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Share inventory records with ownership and borrowing" in starter/src so that the supplied tests pass. Choose &, &mut or by-value parameters to match each function's intent; avoid clone() unless you justify it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 3 / EVIDENCE

Share inventory records with ownership and borrowing / acceptance

✅ Expected result

cargo test reports 5 passed; the overflow test proves the item is unchanged. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

2

TOPIC 2 / K2/A2

AI Vibe Coding for Rust Application Development

Activities 4-6 / concepts, contracts, failure modes and evidence

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / MODEL

The vibe-coding loop for Rust

Read the code, then the contract: what must always be true?

```
1. intent     "parse add/remove/list commands"
2. contract   types, ranges, errors, tests
3. prompt     bounded request to the AI
4. generate   AI proposes code
5. compile    cargo build   (the first reviewer)
6. test       cargo test    (the specification)
7. review     read, question, accept or reject
8. record     decision + evidence
```

Intent

Describe the outcome in plain language, as a user would.

Contract

Pin down types, ranges and errors so the output can be judged.

Compiler + tests

Rust rejects many wrong programs before they run.

Human review

You own the code: accept only what you can explain.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EXACT INTERFACE

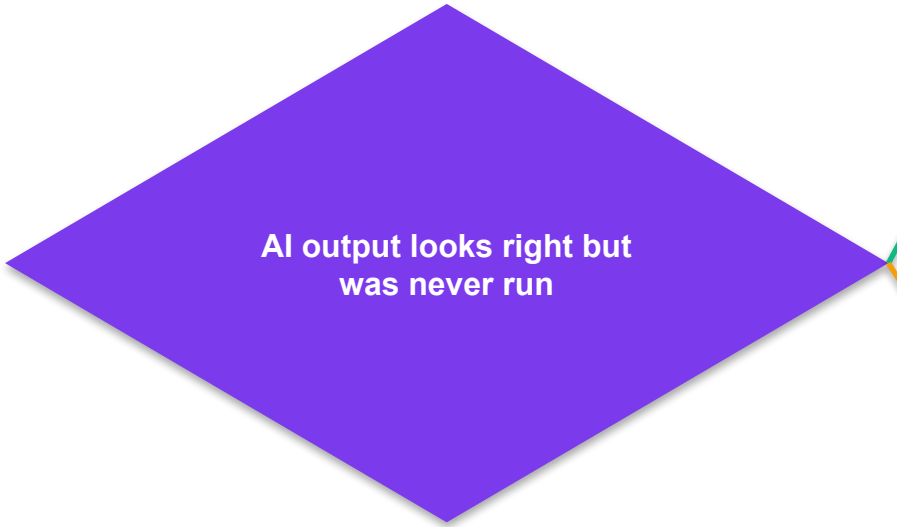
The vibe-coding loop for Rust / contract

Contract field	Exact rule
Speed	AI drafts in seconds; the loop turns drafts into trusted code.
Iteration	Small prompts and small diffs are easier to review.
Stop rule	Never accept code that fails a gate or that you cannot explain.
Record	Keep prompt, proposal, decision and test result for each change.

WHEN IT MATTERS Each activity review record shows prompt, decision and the passing test result.

© 2026 Tertiary Infotech Academy Pte Ltd

The vibe-coding loop for Rust / failure



Symptom present: fix

Run cargo test and clippy before accepting; paste failures back as evidence.

Contract holds: evidence

Each activity review record shows prompt, decision and the passing test result.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / CONFIG

Write a bounded prompt

Read the code, then the contract: what must always be true?

```
Role:      Act as a senior Rust reviewer.
Context:   StockPilot command parser, Rust 2021, std only.
Contract:  parse_command(&[&str]) -> Result<Command, String>
           list | add <sku> <qty> | remove <sku> <qty>
           qty: u32 and >= 1
Limits:    no new crates, no unsafe, no unwrap on input,
           keep the public signature.
Output:    code + 4 boundary tests + one-line reasons.
```

Role

Sets the expected quality bar and tone.

Contract

The exact signature and rules to satisfy.

Limits

What the AI must not do: crates, unsafe, signature changes.

Output

Ask for tests and reasons, not only code.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EXACT INTERFACE

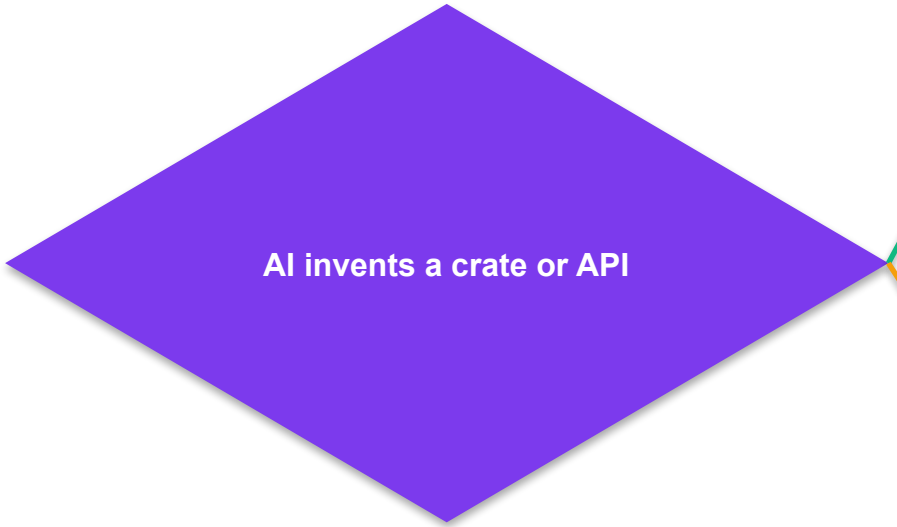
Write a bounded prompt / contract

Contract field	Exact rule
Include	Specification, relevant file, exact cargo output.
Exclude	Secrets, API keys, personal data, assessment answers.
Size	One function or one fix per prompt.
Follow-up	Ask "which edge case did you not test?" before accepting.

WHEN IT MATTERS The accepted proposal compiles, passes the tests and respects every limit.

© 2026 Tertiary Infotech Academy Pte Ltd

Write a bounded prompt / failure



Symptom present: fix

Reject; restate the limits and ask for a std-only version, then compile to confirm.

Contract holds: evidence

The accepted proposal compiles, passes the tests and respects every limit.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EVIDENCE

The compiler is your first reviewer

Read the code, then the contract: what must always be true?

```
error[E0382]: borrow of moved value: `items`
 --> src/main.rs:9:22
  |
5 |     let items = vec![cable, charger];
  |         ----- move occurs because `items` has type
  |               `Vec<Item>`, which does not implement Copy
7 |     let kept = in_stock(items);
  |                         ----- value moved here
9 |     println!("{:?}", items);
  |                      ^^^^^ value borrowed here after move
```

Error code

E0382 can be looked up with rustc --explain E0382.

Spans

The arrows show where the value moved and where it was used.

Help lines

Often suggest the exact fix (borrow, clone, add mut).

AI use

Paste the full message; ask for the cause before the fix.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EXACT INTERFACE

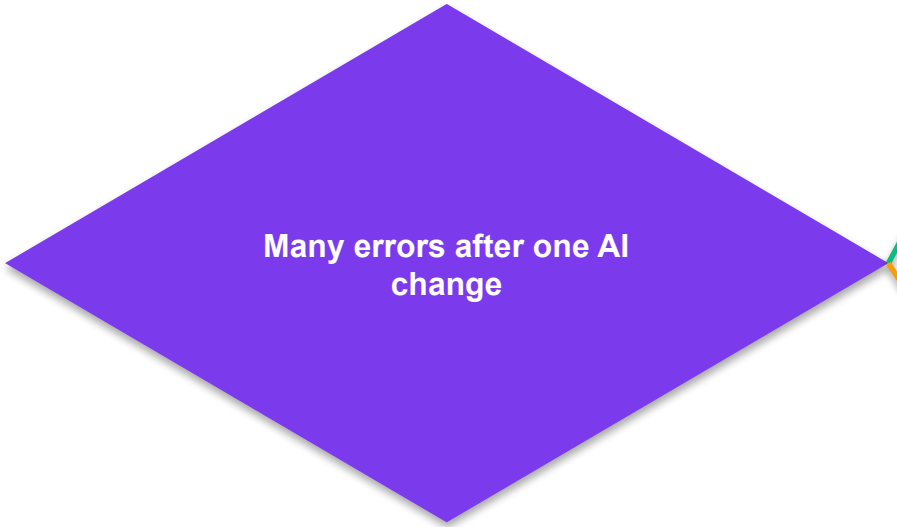
The compiler is your first reviewer / contract

Contract field	Exact rule
Read order	First error first; later errors are often consequences.
Explain	rustc --explain E0XXX gives a worked example.
Fix scope	Smallest change that keeps the contract.
Evidence	Record the error and the fix in the review record.

WHEN IT MATTERS cargo build reports zero errors and zero warnings.

© 2026 Tertiary Infotech Academy Pte Ltd

The compiler is your first reviewer / failure



Symptom present: fix

Revert to the last green state, apply the change in smaller steps and rebuild.

Contract holds: evidence

cargo build reports zero errors and zero warnings.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EVIDENCE

Review AI-generated Rust before accepting it

Read the code, then the contract: what must always be true?

```
// Red flags in a proposal
let qty: u32 = args[2].parse().unwrap();   // panics on bad input
unsafe { ... }                             // never needed here
let copy = items.clone();                  // unjustified clone
use clap::Parser;                          // new crate, not allowed

// Accept when
cargo test                                 # all pass
cargo clippy --all-targets -- -D warnings  # clean
```

unwrap/expect

Acceptable in tests; in library code they turn bad input into a crash.

unsafe

Removes compiler guarantees; not needed for this course.

clone

Often hides an ownership question; ask why it is needed.

New crates

Add supply-chain risk; only with explicit approval.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EXACT INTERFACE

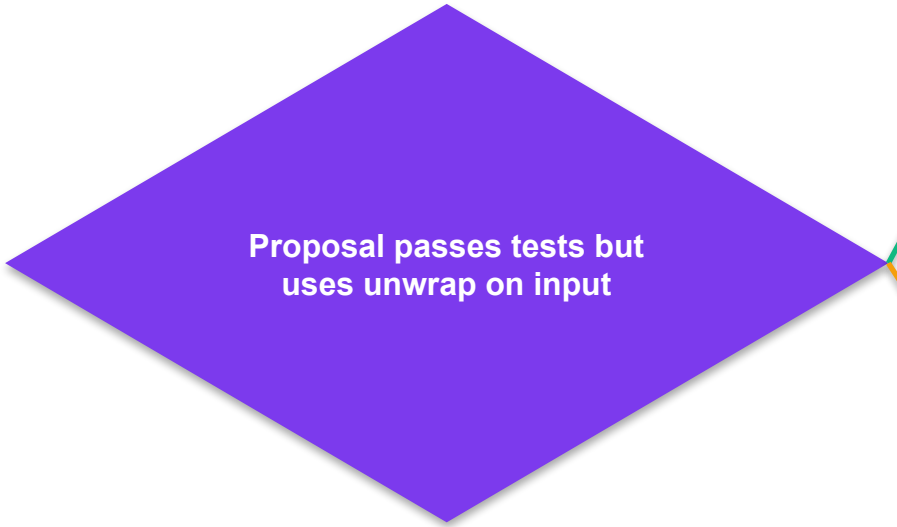
Review AI-generated Rust before accepting it / contract

Contract field	Exact rule
Signatures	Public API unchanged unless the specification changed.
Errors	Invalid input returns Err/None, never panics.
Tests	Every boundary in the specification is tested.
Explainability	You can explain every line to a colleague.

WHEN IT MATTERS The review record lists at least one rejected or edited suggestion with a reason.

© 2026 Tertiary Infotech Academy Pte Ltd

Review AI-generated Rust before accepting it / failure



Symptom present: fix

Reject; ask for map_err/? so bad input returns an error the caller can handle.

Contract holds: evidence

The review record lists at least one rejected or edited suggestion with a reason.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Modules and visibility organise a crate

Read the code, then the contract: what must always be true?

```
// src/lib.rs
pub mod money;      // loads src/money.rs
pub mod report;     // loads src/report.rs

// src/report.rs
use crate::money::format_cents;

pub fn report_line(name: &str, qty: u32, unit: u64)
    -> Option<String> { /* ... */ }
fn helper() {}      // private to this module
```

mod

Declares a module; the file name matches the module name.

Privacy

Everything is private unless marked pub.

Paths

crate:: starts from the crate root; super:: goes up one level.

use

Brings a path into scope so it can be called by its short name.

© 2026 Tertiary Infotech Academy Pte Ltd

Modules and visibility organise a crate / contract

Contract field	Exact rule
pub	Visible to other crates, including your tests/ folder.
pub(crate)	Visible inside this crate only.
Module per concern	money formats currency; report builds lines.
API check	Integration tests compile only against pub items.

WHEN IT MATTERS Activity 5 tests import activity05::money and activity05::report successfully.

© 2026 Tertiary Infotech Academy Pte Ltd

Modules and visibility organise a crate / failure



Symptom present: fix

Make it pub if it is part of the API, or test it inside the module with #[cfg(test)].

Contract holds: evidence

Activity 5 tests import activity05::money and activity05::report successfully.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Cargo dependencies and Cargo.lock

Read the code, then the contract: what must always be true?

```
# Cargo.toml
[dependencies]
serde = { version = "1", features = ["derive"] }
serde_json = "1"

cargo add serde --features derive   # edits Cargo.toml
cargo build                         # downloads + locks
cargo tree                          # show dependency graph
cargo update                        # move within semver range
```

Semver

"1" means any compatible 1.x version.

Features

Opt-in capabilities such as serde's derive macros.

Cargo.lock

Exact versions used; commit it for applications and courses.

crates.io

The public registry; check downloads, docs and maintenance.

© 2026 Tertiary Infotech Academy Pte Ltd

Cargo dependencies and Cargo.lock / contract

Contract field	Exact rule
Approval	Every new crate is a design decision recorded in review.
Reproducibility	Cargo.lock gives every learner the same versions.
Offline	cargo fetch downloads everything before class.
Audit	cargo tree shows what a crate pulls in.

WHEN IT MATTERS Activity 6 builds with serde 1.x recorded in Cargo.lock and 6 tests pass.

© 2026 Tertiary Infotech Academy Pte Ltd

Cargo dependencies and Cargo.lock / failure



Symptom present: fix

Check the version string and network, then cargo fetch; never copy crates by hand.

Contract holds: evidence

Activity 6 builds with serde 1.x recorded in Cargo.lock and 6 tests pass.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

fmt and clippy encode team practice

Read the code, then the contract: what must always be true?

```
cargo fmt                              # rewrite to house style
cargo fmt --check                      # fail if not formatted
cargo clippy --all-targets -- -D warnings

warning: this `if` has identical blocks
warning: redundant clone
warning: missing documentation for a function
  = help: for further information visit
    https://rust-lang.github.io/rust-clippy/...
```

rustfmt

One formatting style for everyone; no style debates in review.

clippy

Hundreds of lints for bugs, performance and idiom.

-D warnings

Treat warnings as errors so they cannot accumulate.

rustfmt.toml

Project settings, for example max_width = 80.

© 2026 Tertiary Infotech Academy Pte Ltd

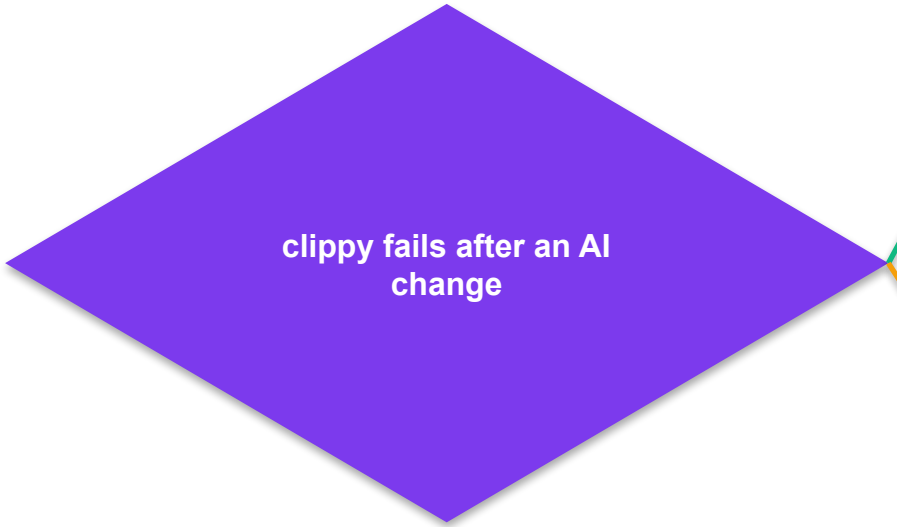
fmt and clippy encode team practice / contract

Contract field	Exact rule
Before commit	fmt --check, clippy -D warnings and test must all pass.
AI output	Run the same gates on generated code; no exceptions.
Allow a lint	Only with #[allow(...)] plus a reason in a comment.
CI	The same three commands can run in a pipeline.

WHEN IT MATTERS Every activity solution passes fmt --check and clippy -D warnings.

© 2026 Tertiary Infotech Academy Pte Ltd

fmt and clippy encode team practice / failure



Symptom present: fix

Read the lint link, apply the suggested idiom, rerun tests to confirm behaviour.

Contract holds: evidence

Every activity solution passes fmt --check and clippy -D warnings.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Set up VS Code for Rust vibe coding

Read the code, then the contract: what must always be true?

```
Extensions
  rust-analyzer            types, errors, go-to-definition
  (optional) CodeLLDB      step-through debugging
  your AI assistant        Copilot / Claude / ChatGPT

Workspace habits
  open the crate folder    (the one with Cargo.toml)
  inlay hints on           see inferred types
  terminal: cargo test     after every accepted change
```

rust-analyzer

Shows the same errors as the compiler while you type.

Inlay hints

Display inferred types, which helps review AI code.

Run/Test lenses

Click "Run Test" above a #[test] function.

AI panel

Keep prompts beside the code; paste exact errors.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / EXACT INTERFACE

Set up VS Code for Rust vibe coding / contract

Contract field	Exact rule
Folder	Open starter/ or solution/ so rust-analyzer finds Cargo.toml.
Save	Format on save can call rustfmt automatically.
Terminal	Use the integrated terminal for cargo commands.
Privacy	Do not paste secrets or assessment answers into AI tools.

WHEN IT MATTERS Hovering a variable shows its type and cargo test runs from the terminal.

© 2026 Tertiary Infotech Academy Pte Ltd

Set up VS Code for Rust vibe coding / failure



Symptom present: fix

Open the folder that contains Cargo.toml, then reload the window.

Contract holds: evidence

Hovering a variable shows its type and cargo test runs from the terminal.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / 25 MIN HANDS-ON

Vibe-code a command parser

ACTIVITY 4

Independent practice; full steps in the activity README and Learner Guide

Store staff will type commands such as "add ABC-0001 5". You describe the command language to an AI assistant, review the generated parser, and keep only code that passes the acceptance tests.

YOU'LL BUILD

labs/activity-04-cli-commands: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 5 passed; cargo run -- add ABC-0001 5 prints Add { sku: "ABC-0001", qty: 5 }.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 4 / LEARN RUST

Vibe-code a command parser / learn it first

samples/01_command_line_args.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: reading command-line arguments.
use std::env;

fn main() {
    let args: Vec<String> = env::args().skip(1).collect();
    if args.is_empty() {
        println!("no arguments: try ./01_command_line_args add ABC-0001 5");
        return;
    }
    for (index, arg) in args.iter().enumerate() {
        println!("argument {index}: {arg}");
    }
}
```

Run the sample

rustc --edition 2021 01_command_line_args.rs && ./01_command_line_args add ABC-0001 5

Explain it simply

Ask the AI to explain reading command-line arguments, slice patterns, parsing text into numbers and returning Result.

Quiz me

5 questions, one at a time, including a predict-the-output question.

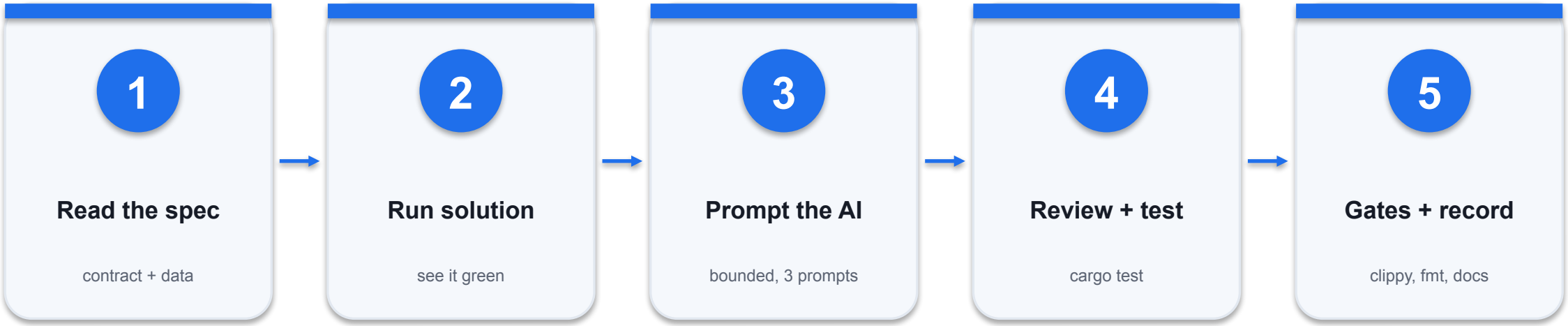
Compare with what I know

Rust vs Python / JavaScript for command-line input and parsing.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 4 / WORKFLOW

Vibe-code a command parser / workflow



EXACT CONTRACT

parse_command(&[&str]) -> Result<Command, String>. Accepts exactly: list | add <sku> <qty> | remove <sku> <qty>. qty must parse as u32 and be at least 1. Anything else returns Err with a usage or reason message.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 4 / CODE

Vibe-code a command parser / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
pub fn parse_command(args: &[&str]) -> Result<Command, String> {
    match args {
        ["list"] => Ok(Command::List),
        [verb @ ("add" | "remove"), sku, qty] => {
            let qty: u32 = qty
                .parse()
                .map_err(|_| format!("invalid quantity: {qty}"))?;
            if qty == 0 {
                return Err("quantity must be at least 1".to_string());
            }
            let sku = sku.to_string();
            if *verb == "add" {
                Ok(Command::Add { sku, qty })
            } else {
                Ok(Command::Remove { sku, qty })
            }
    // ... full code: solution/src/lib.rs
```

Slice patterns

match on [verb, sku, qty] checks the word count and binds words at once.

Or-pattern

"add" | "remove" shares one arm; verb @ keeps the matched word.

map_err

Turns a ParseIntError into a message staff can act on.

Exit code

main returns 2 for a usage error, so scripts can detect failure.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 4 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Vibe-code a command parser / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: parse_command(&[&str]) -> Result<Command, String>. Accepts exactly: list | add <sku> <qty> | remove <sku> <qty>. qty must parse as u32 and be at least 1. Anything else returns Err with a usage or reason message. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Vibe-code a command parser" in starter/src so that the supplied tests pass. Generate the parser from the command grammar only; reject any suggestion that adds crates such as clap. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 4 / EVIDENCE

Vibe-code a command parser / acceptance

✅ Expected result

cargo test reports 5 passed; cargo run -- add ABC-0001 5 prints Add { sku: "ABC-0001", qty: 5 }. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 / 25 MIN HANDS-ON

Organise a crate into modules

ACTIVITY 5

Independent practice; full steps in the activity README and Learner Guide

The reporting code has grown. You split StockPilot formatting into a money module and a report module, expose only what callers need, and keep the behaviour identical under test.

YOU'LL BUILD

labs/activity-05-modules-visibility: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 4 passed; tests import activity05::money and activity05::report.

© 2026 Tertiary Infotech Academy Pte Ltd

Organise a crate into modules / learn it first

samples/01_inline_modules.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: modules and privacy in one file.
mod money {
    pub fn format_cents(cents: u64) -> String {
        format!("${}.{:02}", cents / 100, cents % 100)
    }

    fn secret_rule() -> &'static str {
        "private: only code inside `money` can call this"
    }

    pub fn explain() -> &'static str {
        secret_rule()
    }
}

mod report {
// ... full script: samples/01_inline_modules.rs
```

Run the sample

rustc --edition 2021 01_inline_modules.rs && ./01_inline_modules

Explain it simply

Ask the AI to explain modules, pub visibility, paths with crate::, self:: and super::, and use ... as.

Quiz me

5 questions, one at a time, including a predict-the-output question.

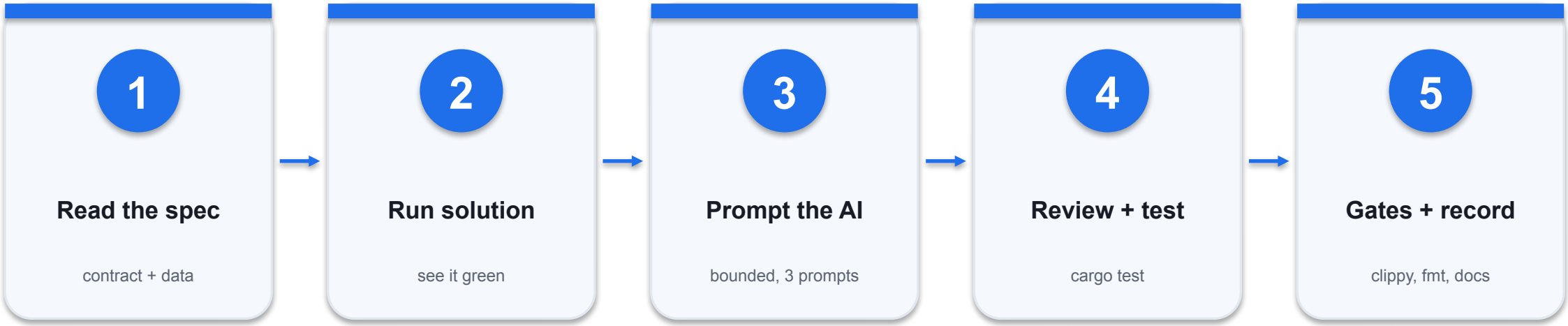
Compare with what I know

Rust vs Python / JavaScript for modules and code organisation.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 5 / WORKFLOW

Organise a crate into modules / workflow



EXACT CONTRACT

money::format_cents(u64) -> String renders "$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 5 / CODE

Organise a crate into modules / Rust

Reference solution: solution/src/report.rs (formatted, tested, clippy-clean)

```rust
/// Builds `"<name> x<qty> @ <unit> = <total>"`, or `None` on overflow.
pub fn report_line(name: &str, qty: u32, unit_cents: u64) -> Option<String> {
    let total = unit_cents.checked_mul(u64::from(qty))?;
    Some(format!(
        "{name} x{qty} @ {} = {}",
        format_cents(unit_cents),
        format_cents(total)
    ))
}
```

mod

pub mod money; loads src/money.rs as the module crate::money.

pub

Items are private by default; pub is a deliberate API decision.

use crate::

report imports format_cents by its full path inside the crate.

collect

Collecting Option<Vec<_>> stops at the first None.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 5 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Organise a crate into modules / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: money::format_cents(u64) -> String renders "$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Organise a crate into modules" in starter/src so that the supplied tests pass. Keep the two-module layout and public paths; do not merge modules to make tests pass. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 5 / EVIDENCE

Organise a crate into modules / acceptance

✅ Expected result

cargo test reports 4 passed; tests import activity05::money and activity05::report. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K2/A2 K4/A4 / 25 MIN HANDS-ON

Load inventory JSON with Cargo dependencies

ACTIVITY 6

Independent practice; full steps in the activity README and Learner Guide

The purchasing team exports inventory as JSON. You add serde and serde_json with Cargo, derive the data contract, and let the type system reject malformed records at the boundary.

YOU'LL BUILD

labs/activity-06-cargo-serde-json: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 6 passed; stock value of data/inventory.json is 65,948 cents.

© 2026 Tertiary Infotech Academy Pte Ltd

Load inventory JSON with Cargo dependencies / learn it first

samples/01_manual_parsing.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: parsing "sku,qty" by hand - the work serde automates for JSON.
struct Item {
    sku: String,
    qty: u32,
}

fn parse(line: &str) -> Result<Item, String> {
    let (sku, qty) = line.split_once(',').ok_or("expected sku,qty")?;
    let qty = qty.trim().parse::<u32>().map_err(|e| format!("qty: {e}"))?;
    Ok(Item {
        sku: sku.trim().to_string(),
        qty,
    })
}

fn main() {
// ... full script: samples/01_manual_parsing.rs
```

Run the sample

rustc --edition 2021 01_manual_parsing.rs && ./01_manual_parsing

Explain it simply

Ask the AI to explain Cargo dependencies, Cargo.lock, parsing structured text and what serde automates.

Quiz me

5 questions, one at a time, including a predict-the-output question.

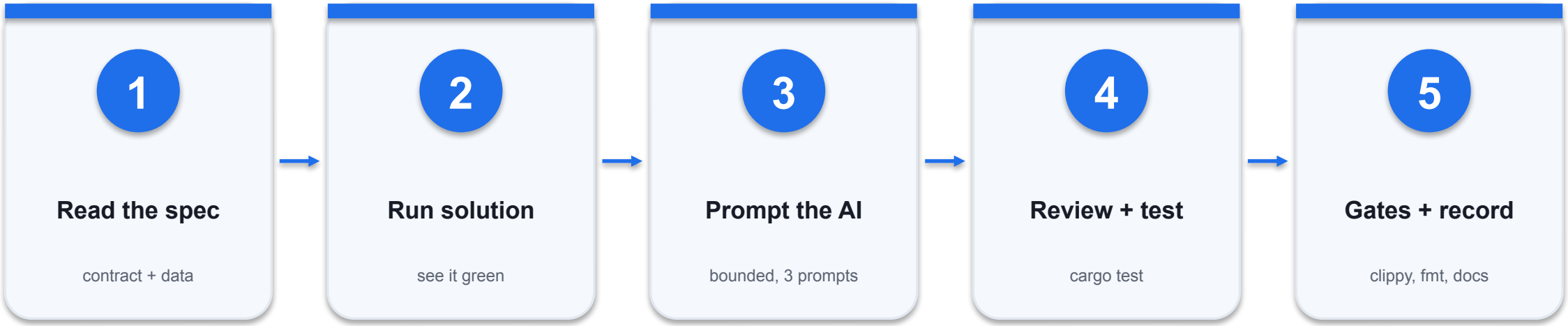
Compare with what I know

Rust vs Python / JavaScript for packages, dependencies and data formats.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 6 / WORKFLOW

Load inventory JSON with Cargo dependencies / workflow



EXACT CONTRACT

Item { sku, name, qty: u32, unit_cents: u64 } derives Serialize and Deserialize. load_items(&str) -> Result<Vec<Item>, serde_json::Error>; to_json(&[Item]) -> Result<String, _>; stock_value(&[Item]) -> Option<u64>.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 6 / CODE

Load inventory JSON with Cargo dependencies / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Parses a JSON array of items; missing or mistyped fields are
errors.
pub fn load_items(json: &str) -> Result<Vec<Item>, serde_json::Error>
{
    serde_json::from_str(json)
}
```

Cargo.toml

serde with the derive feature and serde_json are declared dependencies.

derive

#[derive(Deserialize)] generates the parser from the struct definition.

Types as validation

qty: u32 makes -1 a parse error before business logic runs.

Cargo.lock

Records the exact resolved versions so every learner builds the same code.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 6 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Load inventory JSON with Cargo dependencies / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: Item { sku, name, qty: u32, unit_cents: u64 } derives Serialize and Deserialize. load_items(&str) -> Result<Vec<Item>, serde_json::Error>; to_json(&[Item]) -> Result<String, _>; stock_value(&[Item]) -> Option<u64>. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Load inventory JSON with Cargo dependencies" in starter/src so that the supplied tests pass. Use serde derive only; do not hand-write JSON parsing or add other crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 6 / EVIDENCE

Load inventory JSON with Cargo dependencies / acceptance

✅ Expected result

cargo test reports 6 passed; stock value of data/inventory.json is 65,948 cents. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

3

TOPIC 3 / K3/A3

Rust Programming Controls, Functions and Features

Activities 7-9 / concepts, contracts, failure modes and evidence

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / CODE

if and match are expressions

Read the code, then the contract: what must always be true?

```
let label = if qty == 0 { "out" } else { "in stock" };

let status = match qty {
    0 => "out of stock",
    1..=4 => "low",
    5..=99 => "ok",
    _ => "bulk",
};
```

Expression

if and match produce a value you can bind with let.

Same type

Every branch must return the same type.

Ranges

1..=4 matches 1, 2, 3 and 4 inclusive.

Wildcard

_ matches everything else; use it deliberately.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / EXACT INTERFACE

if and match are expressions / contract

Contract field	Exact rule
Boolean	Conditions must be bool; there is no truthy integer.
Exhaustive	match must cover every possible value.
Order	Arms are tried top to bottom; the first match wins.
Tests	Test each range boundary: 0, 1, 4, 5, 99, 100.

WHEN IT MATTERS Boundary tests cover every arm.

© 2026 Tertiary Infotech Academy Pte Ltd

if and match are expressions / failure



Symptom present: fix

Add the missing arm (the compiler lists it) instead of a blanket _ that hides it.

Contract holds: evidence

Boundary tests cover every arm.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / CODE

enum + match model every case

Read the code, then the contract: what must always be true?

```
enum Movement { Receive(u32), Ship(u32), Adjust(i64) }

match movement {
    Movement::Receive(q) => stock.checked_add(q)
        .ok_or(StockError::Overflow),
    Movement::Ship(q) if q > stock => Err(Insufficient {..}),
    Movement::Ship(q) => Ok(stock - q),
    Movement::Adjust(d) => adjust(stock, d),
}
```

Variants with data

Each variant carries exactly the data it needs.

Guards

if q > stock refines an arm with a condition.

Compiler help

Adding a variant makes every incomplete match fail to compile.

Errors as enums

StockError names each failure for callers and tests.

© 2026 Tertiary Infotech Academy Pte Ltd

enum + match model every case / contract

Contract field	Exact rule
Receive	checked_add; Overflow on u32 overflow.
Ship	Insufficient when requested > available.
Adjust	NegativeResult below zero; Overflow above u32.
Sequence	apply_all stops at the first error.

WHEN IT MATTERS Activity 7: six tests cover every variant and every error.

© 2026 Tertiary Infotech Academy Pte Ltd

enum + match model every case / failure



Symptom present: fix

Avoid _ in matches over your own enums so the compiler lists every place to update.

Contract holds: evidence

Activity 7: six tests cover every variant and every error.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Loops: for, while and loop

Read the code, then the contract: what must always be true?

```
for item in &items {                    // iterate by reference
    total += u64::from(item.qty);
}

for (index, line) in text.lines().enumerate() { /* .. */ }

let mut attempts = 0;
let code = loop {                       // loop can return a value
    attempts += 1;
    if attempts == 3 { break "gave up"; }
};
```

for

Iterates over anything that implements IntoIterator.

&items

Borrows each element; items stays usable afterwards.

enumerate

Adds a 0-based index; add 1 for human line numbers.

loop/break

An infinite loop that can return a value with break.

© 2026 Tertiary Infotech Academy Pte Ltd

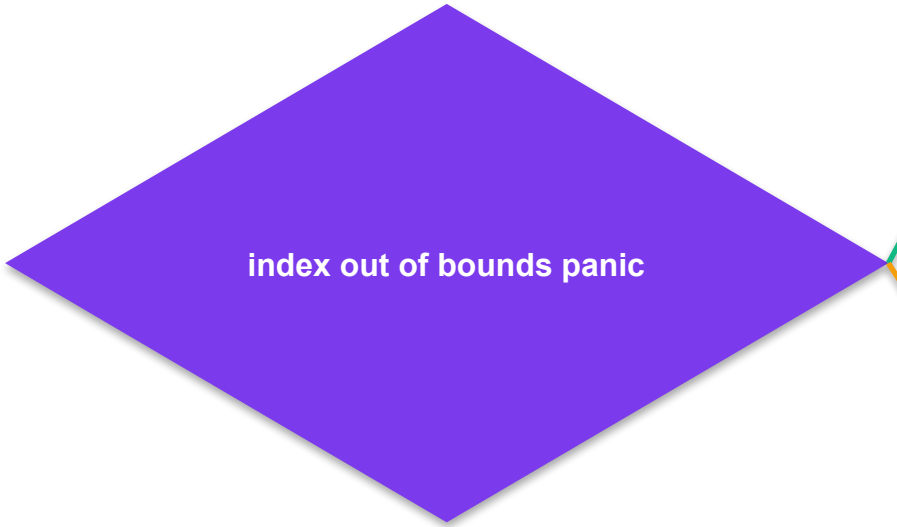
Loops: for, while and loop / contract

Contract field	Exact rule
Preferred	for over iterators; no manual index bounds to get wrong.
while	When the stop condition is not a collection end.
continue	Skip blank lines and headers early.
Early exit	? inside a loop returns the first error from the function.

WHEN IT MATTERS Activity 10 numbers errors from 1 and skips header and blank lines.

© 2026 Tertiary Infotech Academy Pte Ltd

Loops: for, while and loop / failure



Symptom present: fix

Iterate with for x in &v or use v.get(i) which returns Option.

Contract holds: evidence

Activity 10 numbers errors from 1 and skips header and blank lines.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / CODE

Structs and impl blocks

Read the code, then the contract: what must always be true?

```rust
#[derive(Debug, Default)]
pub struct Inventory {
    stock: HashMap<String, u32>,    // private field
}

impl Inventory {
    pub fn new() -> Self { Self::default() }
    pub fn quantity(&self, sku: &str) -> u32 { .. }
    pub fn ship(&mut self, sku: &str, qty: u32)
        -> Result<u32, String> { .. }
}
```

Struct

Groups related data under one named type.

Private fields

Force every change through methods that keep the rules.

&self / &mut self

Methods declare whether they read or change the value.

derive

Debug, Clone, PartialEq, Default generated by the compiler.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / EXACT INTERFACE

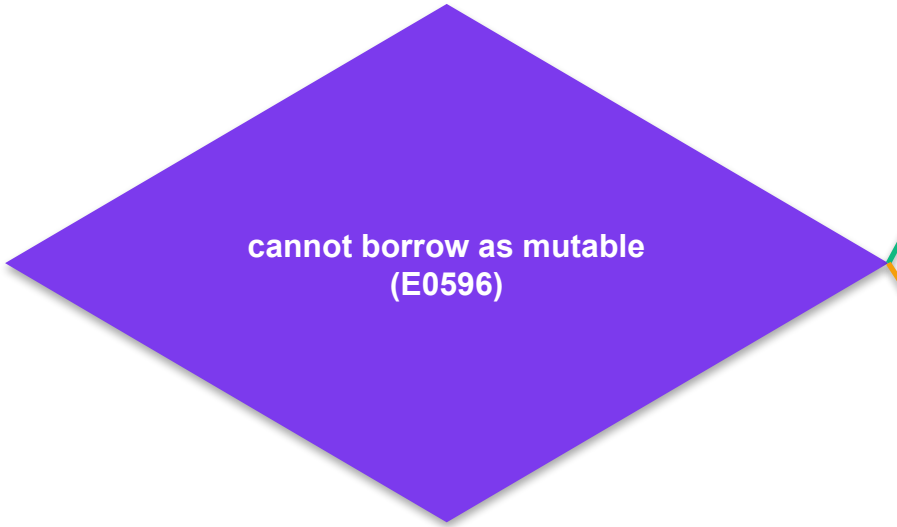
Structs and impl blocks / contract

Contract field	Exact rule
Constructor	new() by convention; Default for empty values.
Readers	&self methods such as quantity and low_stock.
Writers	&mut self methods such as receive and ship.
Invariant	Stock never goes below zero because only ship can reduce it.

WHEN IT MATTERS Activity 8: callers cannot touch the HashMap directly and six tests pass.

© 2026 Tertiary Infotech Academy Pte Ltd

Structs and impl blocks / failure



Symptom present: fix

Declare the variable with let mut, or change the method to take &self if it only reads.

Contract holds: evidence

Activity 8: callers cannot touch the HashMap directly and six tests pass.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Collections: Vec and HashMap

Read the code, then the contract: what must always be true?

```
let mut skus: Vec<String> = Vec::new();
skus.push("ABC-0001".to_string());
skus.sort();

let mut stock: HashMap<String, u32> = HashMap::new();
*stock.entry("ABC-0001".into()).or_insert(0) += 5;
let level = stock.get("ABC-0001").copied().unwrap_or(0);

if let Some(q) = stock.get_mut("ABC-0001") { *q -= 1; }
```

Vec<T>

Growable, ordered list; index or iterate.

HashMap<K, V>

Key lookup in constant average time; no order.

entry API

Insert-or-update with a single lookup.

get returns Option

A missing key is a normal case, not a crash.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / EXACT INTERFACE

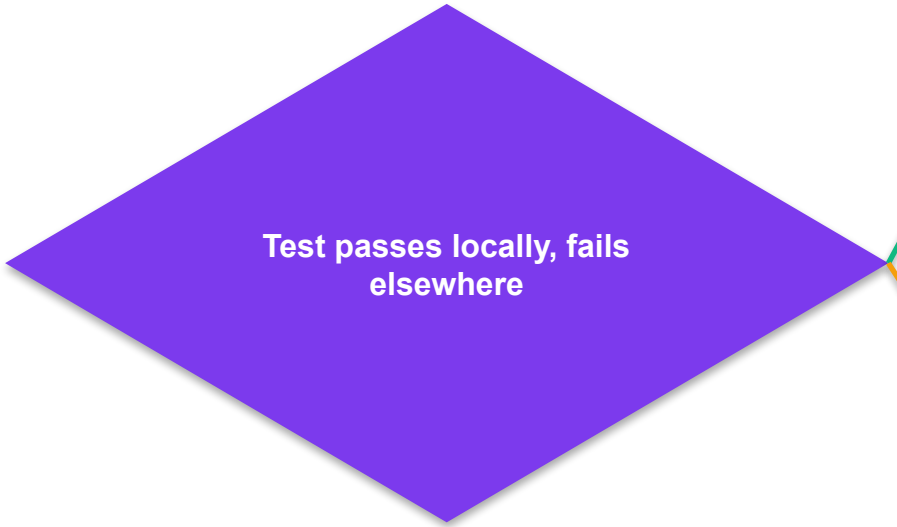
Collections: Vec and HashMap / contract

Contract field	Exact rule
Order	Sort before output because HashMap order is random.
Ownership	Keys are owned Strings; look up with &str.
Missing key	unwrap_or(0) or an explicit error, never map[key] panic.
Choice	Vec for sequences; HashMap for lookup by id.

WHEN IT MATTERS low_stock returns a sorted Vec so tests are deterministic.

© 2026 Tertiary Infotech Academy Pte Ltd

Collections: Vec and HashMap / failure



Symptom present: fix

Output depended on HashMap order; sort the result before comparing.

Contract holds: evidence

low_stock returns a sorted Vec so tests are deterministic.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Traits and generics share behaviour

Read the code, then the contract: what must always be true?

```
pub trait Priced {
    fn unit_cents(&self) -> u64;
    fn quantity(&self) -> u32;
    fn line_cents(&self) -> Option<u64> {       // default
        self.unit_cents().checked_mul(u64::from(self.quantity()))
    }
}
impl Priced for Product { .. }
impl Priced for Service { .. }

fn order_total<T: Priced>(lines: &[T]) -> Option<u64>
fn mixed_total(lines: &[&dyn Priced]) -> Option<u64>
```

Trait

A named set of methods a type promises to provide.

Default method

Written once, inherited by every implementor.

Generic T: Priced

One type per call, resolved at compile time (fast).

&dyn Priced

Different types in one list, resolved at run time.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / EXACT INTERFACE

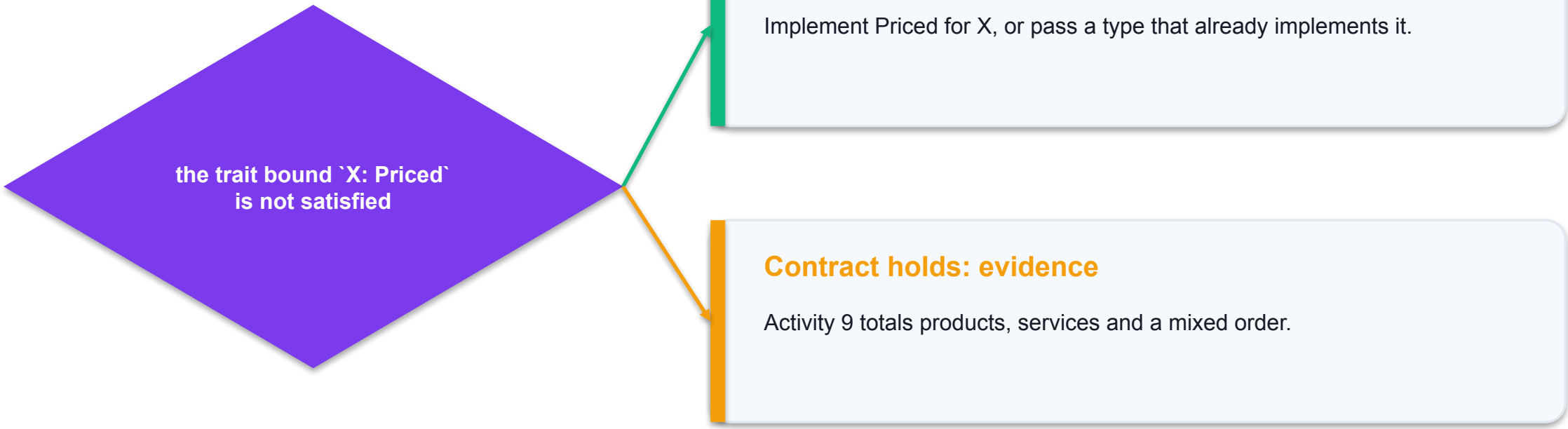
Traits and generics share behaviour / contract

Contract field	Exact rule
Contract	Implementors supply unit_cents and quantity only.
Extension	A new type joins by implementing the trait; totals are unchanged.
Dispatch	Generics for speed; trait objects for mixed collections.
Tests	Test the default method once via any implementor.

WHEN IT MATTERS Activity 9 totals products, services and a mixed order.

© 2026 Tertiary Infotech Academy Pte Ltd

Traits and generics share behaviour / failure



Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / CODE

Closures and iterator chains

Read the code, then the contract: what must always be true?

```
let expensive: Vec<&str> = products
    .iter()
    .filter(|p| p.unit_cents >= 1_000)
    .map(|p| p.name.as_str())
    .collect();

let total = lines.iter().try_fold(0u64, |acc, line| {
    acc.checked_add(line.line_cents()?)
});
```

Closure

|p| ... is an anonymous function that can capture variables.

Lazy

filter and map do nothing until collect or a fold consumes them.

collect

The target type (Vec, String, Option<Vec>) decides the result.

try_fold

A fold that stops at the first None or Err.

© 2026 Tertiary Infotech Academy Pte Ltd

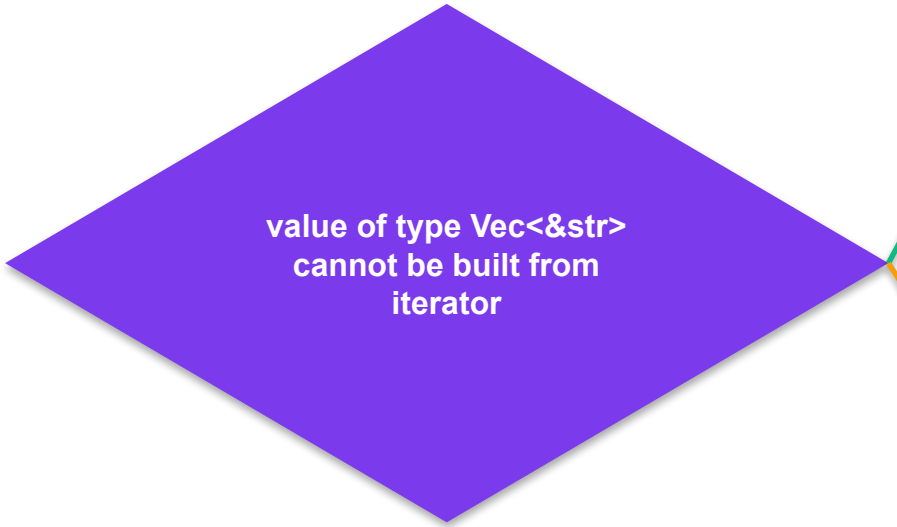
Closures and iterator chains / contract

Contract field	Exact rule
Readability	Each step names one transformation.
Safety	No manual indexes, so no out-of-bounds errors.
Performance	Compiles to the same machine code as a hand loop.
Order	Iterator chains keep input order unless you sort.

WHEN IT MATTERS names_at_least keeps input order; try_fold returns None on overflow.

© 2026 Tertiary Infotech Academy Pte Ltd

Closures and iterator chains / failure



Symptom present: fix

Check what map returns (&str vs String) and annotate the collect target type.

Contract holds: evidence

names_at_least keeps input order; try_fold returns None on overflow.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / CODE

Option and Result are control features

Read the code, then the contract: what must always be true?

```
fn find(sku: &str) -> Option<u32>              // may be absent
fn ship(sku: &str, q: u32) -> Result<u32, String> // may fail

match find("ABC-0001") {
    Some(q) => println!("{q} on hand"),
    None => println!("unknown SKU"),
}
let q = find("ABC-0001").unwrap_or(0);
let left = ship("ABC-0001", 2)?;              // propagate
```

Option<T>

Some(value) or None: absence without null.

Result<T, E>

Ok(value) or Err(reason): failure without exceptions.

Combinators

map, and_then, ok_or, unwrap_or transform without match.

?

Returns early with the None/Err, otherwise unwraps the value.

© 2026 Tertiary Infotech Academy Pte Ltd

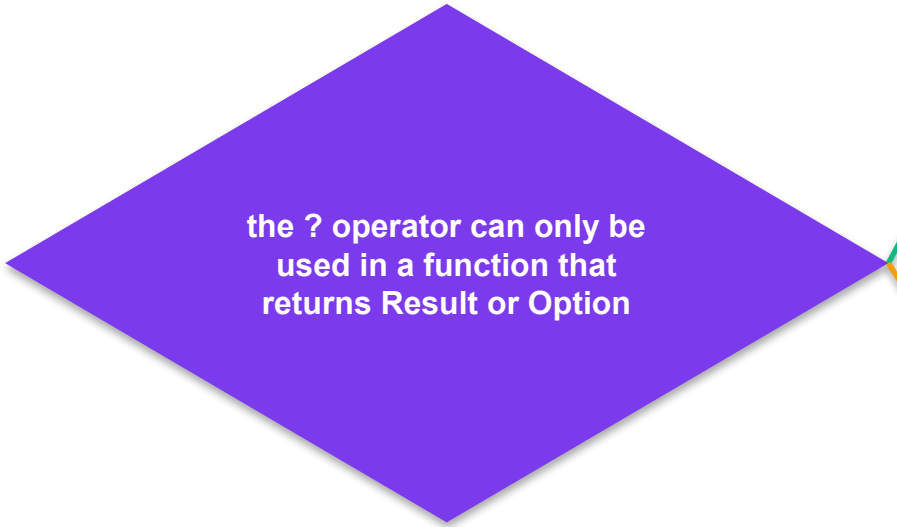
Option and Result are control features / contract

Contract field	Exact rule
Absent	Use Option when there is no reason to report.
Failed	Use Result when the caller needs to know why.
Library code	Return Option/Result; let the caller decide.
Binaries	Convert errors to messages and exit codes at the edge.

WHEN IT MATTERS No unwrap() on input data remains in any solution crate.

© 2026 Tertiary Infotech Academy Pte Ltd

Option and Result are control features / failure



Symptom present: fix

Change the function return type, or handle the value with match.

Contract holds: evidence

No unwrap() on input data remains in any solution crate.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / 30 MIN HANDS-ON

Apply stock movements with enums and match

ACTIVITY 7

Independent practice; full steps in the activity README and Learner Guide

Stock changes through receipts, shipments and manual adjustments. An enum names every movement and match forces each one to be handled, including the failure cases.

YOU'LL BUILD

labs/activity-07-match-control-flow: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 6 passed, covering every variant and every error.

© 2026 Tertiary Infotech Academy Pte Ltd

Apply stock movements with enums and match / learn it first

samples/01_enum_match.rs: predict the output, run it, then use the Learn Rust prompts

```
// Sample: an enum with data and an exhaustive match with a guard.
enum Movement {
    Receive(u32),
    Ship(u32),
    Adjust(i64),
}

fn describe(movement: &Movement) -> String {
    match movement {
        Movement::Receive(qty) => format!("receive {qty}"),
        Movement::Ship(qty) if *qty > 100 => {
            format!("ship {qty} (large order, needs approval)")
        }
        Movement::Ship(qty) => format!("ship {qty}"),
        Movement::Adjust(delta) => format!("adjust by {delta:+}"),
    }
// ... full script: samples/01_enum_match.rs
```

Run the sample

rustc --edition 2021 01_enum_match.rs && ./01_enum_match

Explain it simply

Ask the AI to explain enums with data, match with guards, if let, ranges in patterns and loops.

Quiz me

5 questions, one at a time, including a predict-the-output question.

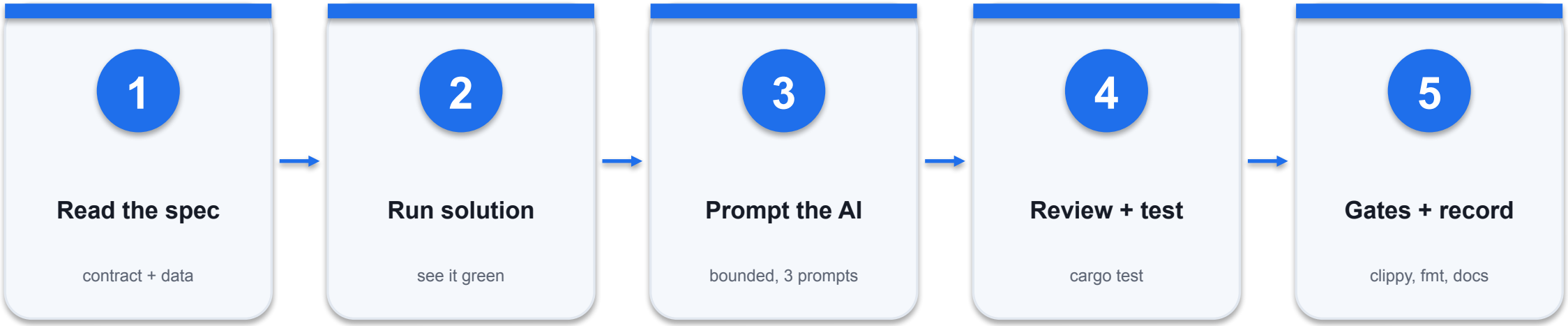
Compare with what I know

Rust vs Python / JavaScript for control flow: enum, match, if and loops.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 7 / WORKFLOW

Apply stock movements with enums and match / workflow



EXACT CONTRACT

Movement::{Receive(u32), Ship(u32), Adjust(i64)}. apply(stock, movement) -> Result<u32, StockError>: shipping more than available is Insufficient; results below zero are NegativeResult; above u32 are Overflow. apply_all stops at the first error.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 7 / CODE

Apply stock movements with enums and match / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
pub fn apply(stock: u32, movement: Movement) -> Result<u32, StockError> {
    match movement {
        Movement::Receive(qty) => {
            stock.checked_add(qty).ok_or(StockError::Overflow)
        }
        Movement::Ship(qty) if qty > stock => Err(StockError::Insufficient {
            available: stock,
            requested: qty,
        }),
        Movement::Ship(qty) => Ok(stock - qty),
        Movement::Adjust(delta) => {
            let next = i64::from(stock)
                .checked_add(delta)
                .ok_or(StockError::Overflow)?;
            if next < 0 {
                return Err(StockError::NegativeResult);
    // ... full code: solution/src/lib.rs
```

enum

Each movement kind is a variant that can carry its own data.

match

The compiler rejects a match that forgets a variant.

Guard

Ship(qty) if qty > stock is tested before the normal Ship arm.

? in a loop

apply_all returns the first Err without applying later moves.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 7 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Apply stock movements with enums and match / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: Movement::{Receive(u32), Ship(u32), Adjust(i64)}. apply(stock, movement) -> Result<u32, StockError>: shipping more than available is Insufficient; results below zero are NegativeResult; above u32 are Overflow. apply_all stops at the first error. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Apply stock movements with enums and match" in starter/src so that the supplied tests pass. Use enum + match with exhaustive arms; no wildcard _ arm that hides future variants. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 7 / EVIDENCE

Apply stock movements with enums and match / acceptance

✅ Expected result

cargo test reports 6 passed, covering every variant and every error. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / 30 MIN HANDS-ON

Track stock in a HashMap

ACTIVITY 8

Independent practice; full steps in the activity README and Learner Guide

StockPilot needs fast lookup by SKU. You wrap a HashMap in an Inventory type so every change goes through methods that enforce the stock rules.

YOU'LL BUILD

labs/activity-08-hashmap-inventory: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 6 passed; low_stock(5) returns AAA-0001 and BBB-0002 only.

© 2026 Tertiary Infotech Academy Pte Ltd

Track stock in a HashMap / learn it first

samples/01_vec_basics.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: a growable Vec, sorting, safe access with get.
fn main() {
    let mut skus: Vec<String> = Vec::new();
    skus.push("CCC-0003".to_string());
    skus.push("AAA-0001".to_string());
    skus.push("BBB-0002".to_string());
    skus.sort();
    println!("sorted: {skus:?}");
    println!("first: {:?}, tenth: {:?}", skus.first(), skus.get(9));
    for (i, sku) in skus.iter().enumerate() {
        println!("{}. {sku}", i + 1);
    }
}
```

Run the sample

rustc --edition 2021 01_vec_basics.rs && ./01_vec_basics

Explain it simply

Ask the AI to explain Vec, HashMap, the entry API, Option from get, structs and sorting for deterministic output.

Quiz me

5 questions, one at a time, including a predict-the-output question.

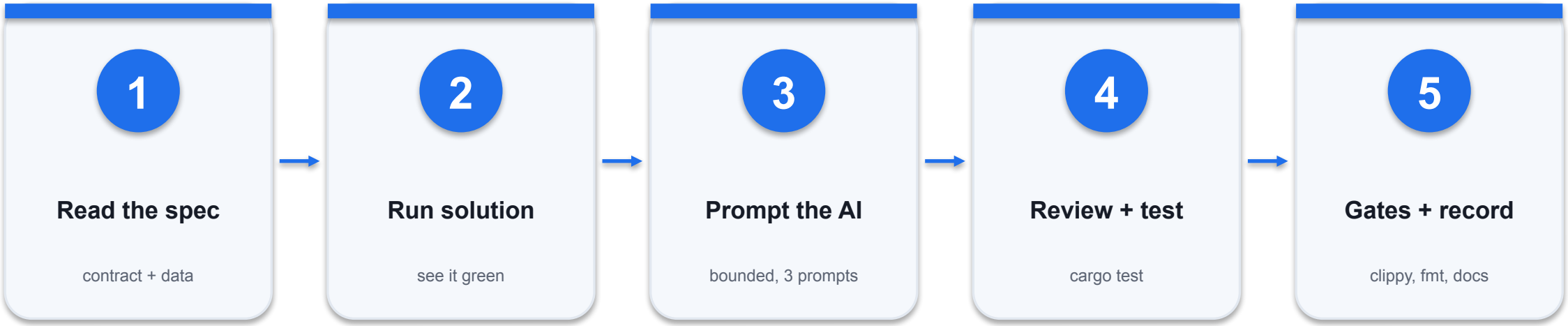
Compare with what I know

Rust vs Python / JavaScript for collections (lists and dictionaries).

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 8 / WORKFLOW

Track stock in a HashMap / workflow



EXACT CONTRACT

Inventory::receive(&mut self, sku, qty) -> Option<u32> (None on overflow, unchanged); ship -> Result<u32, String> (unknown SKU or insufficient stock is Err, unchanged); quantity(sku) -> u32 (0 when unknown); low_stock(threshold) -> Vec<(String, u32)> strictly below threshold, sorted by SKU.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 8 / CODE

Track stock in a HashMap / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Removes units; unknown SKUs and shortages are errors.
pub fn ship(&mut self, sku: &str, qty: u32) -> Result<u32, String> {
    let level = self
        .stock
        .get_mut(sku)
        .ok_or_else(|| format!("unknown SKU {sku}"))?;
    if qty > *level {
        return Err(format!("only {level} of {sku} available"));
    }
    *level -= qty;
    Ok(*level)
}
```

Encapsulation

The HashMap field is private; callers can only use the methods.

entry API

entry(..).or_insert(0) creates a missing key in one lookup.

get_mut

A mutable reference into the map lets ship change the level in place.

Sorting

HashMap order is random, so low_stock sorts before returning.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 8 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Track stock in a HashMap / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: Inventory::receive(&mut self, sku, qty) -> Option<u32> (None on overflow, unchanged); ship -> Result<u32, String> (unknown SKU or insufficient stock is Err, unchanged); quantity(sku) -> u32 (0 when unknown); low_stock(threshold) -> Vec<(String, u32)> strictly below threshold, sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Track stock in a HashMap" in starter/src so that the supplied tests pass. Keep the HashMap private and the method signatures unchanged; no unwrap() on lookups. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 8 / EVIDENCE

Track stock in a HashMap / acceptance

✅ Expected result

cargo test reports 6 passed; low_stock(5) returns AAA-0001 and BBB-0002 only. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K3/A3 / 30 MIN HANDS-ON

Price mixed order lines with traits and iterators

ACTIVITY 9

Independent practice; full steps in the activity README and Learner Guide

Orders can contain products and services (for example, device set-up hours). A Priced trait gives both one pricing contract, and iterator chains total and filter them without manual index loops.

YOU'LL BUILD

labs/activity-09-traits-iterators: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 5 passed; the mixed order totals 14,297 cents.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 9 / LEARN RUST

Price mixed order lines with traits and iterators / learn it first

samples/01_trait_basics.rs: predict the output, run it, then use the Learn Rust prompts

```
// Sample: one trait, two types, generic and dynamic dispatch.
trait Priced {
    fn unit_cents(&self) -> u64;
    fn quantity(&self) -> u32;
    fn line_cents(&self) -> u64 {
        self.unit_cents() * u64::from(self.quantity())
    }
}

struct Product {
    unit_cents: u64,
    qty: u32,
}

struct Service {
    rate_cents: u64,
// ... full script: samples/01_trait_basics.rs
```

Run the sample

rustc --edition 2021 01_trait_basics.rs && ./01_trait_basics

Explain it simply

Ask the AI to explain traits, default methods, generics, trait objects (dyn), closures and iterator adaptors.

Quiz me

5 questions, one at a time, including a predict-the-output question.

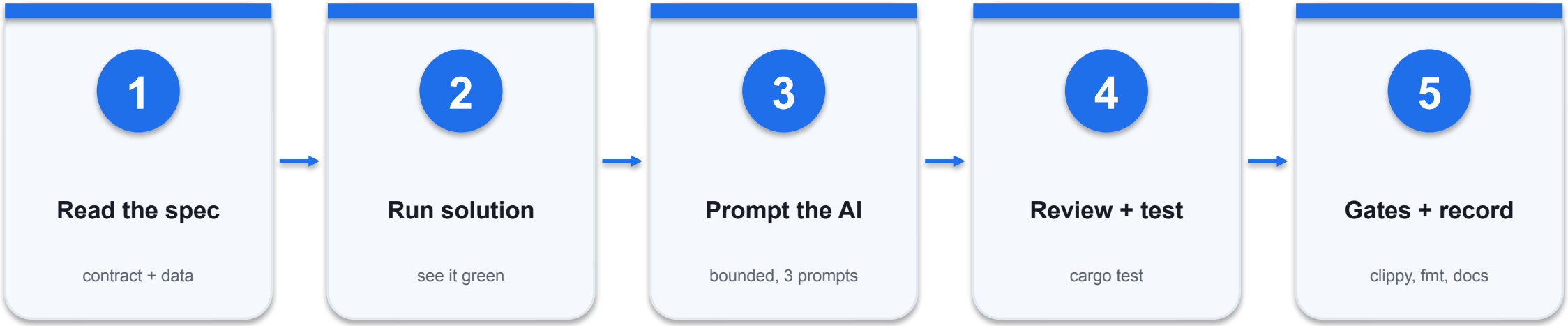
Compare with what I know

Rust vs Python / JavaScript for traits, generics and iterators.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 9 / WORKFLOW

Price mixed order lines with traits and iterators / workflow



EXACT CONTRACT

trait Priced { unit_cents, quantity, line_cents (default, checked) }. order_total<T: Priced>(&[T]) and mixed_total(&[&dyn Priced]) return Option<u64> (None on overflow; empty is Some(0)). names_at_least(&[Product], min_cents) -> Vec<&str> in input order.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 9 / CODE

Price mixed order lines with traits and iterators / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Totals lines of one type (static dispatch).
pub fn order_total<T: Priced>(lines: &[T]) -> Option<u64> {
    lines
        .iter()
        .try_fold(0u64, |total, line| total.checked_add(line.line_cents()?))
}
```

Trait

Priced is a contract: any type that implements it can be totalled.

Default method

line_cents is written once and reused by Product and Service.

Generic vs dyn

T: Priced is resolved at compile time; &dyn Priced at run time.

try_fold

Accumulates a total and stops at the first overflow (None).

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 9 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Price mixed order lines with traits and iterators / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: trait Priced { unit_cents, quantity, line_cents (default, checked) }. order_total<T: Priced>(&[T]) and mixed_total(&[&dyn Priced]) return Option<u64> (None on overflow; empty is Some(0)). names_at_least(&[Product], min_cents) -> Vec<&str> in input order. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Price mixed order lines with traits and iterators" in starter/src so that the supplied tests pass. Implement with iterator adaptors (iter, filter, map, try_fold); no index-based for loops. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 9 / EVIDENCE

Price mixed order lines with traits and iterators / acceptance

✅ Expected result

cargo test reports 5 passed; the mixed order totals 14,297 cents. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

END OF DAY 1

Day 1 recap: the compiler and tests are the gate.

You built typed components, organised them in a crate, and used AI drafts only after they compiled, passed and made sense.

© 2026 Tertiary Infotech Academy Pte Ltd

4

TOPIC 4 / K4/A4

Testing, Debugging and Integrating Rust Components

Activities 10-12 / concepts, contracts, failure modes and evidence

© 2026 Tertiary Infotech Academy Pte Ltd

Unit tests live beside the code

Read the code, then the contract: what must always be true?

```rust
pub fn needs_reorder(on_hand: u32, point: u32) -> bool {
    on_hand <= point
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn boundary_is_inclusive() {
        assert!(needs_reorder(80, 80));
        assert!(!needs_reorder(81, 80));
    }
}
```

#[test]

Marks a function that cargo test runs.

#[cfg(test)]

Compiled only for tests; not shipped in the binary.

use super::*

Unit tests can reach private items of their module.

assert_eq!

Shows left and right values when it fails.

© 2026 Tertiary Infotech Academy Pte Ltd

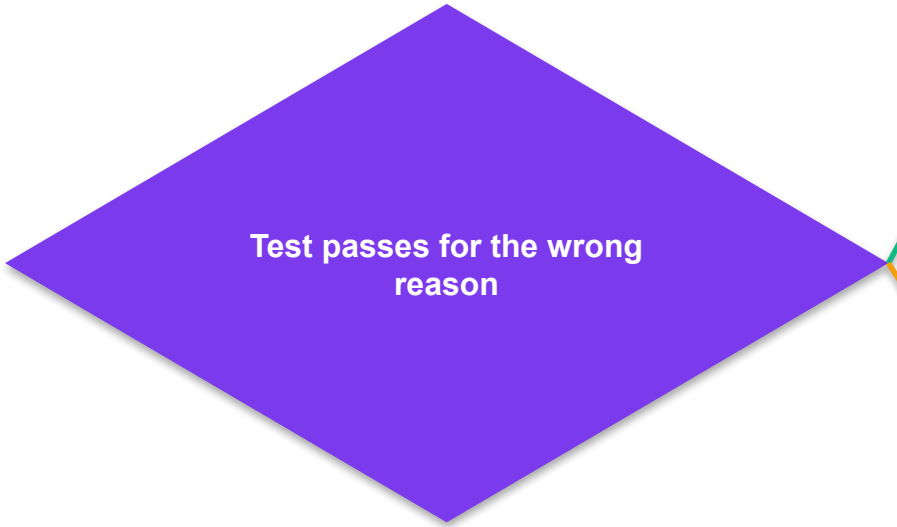
Unit tests live beside the code / contract

Contract field	Exact rule
Name	Describe the rule: boundary_is_inclusive, not test1.
One behaviour	Each test checks one rule so failures are specific.
Boundaries	Test at, just below and just above every limit.
Speed	Unit tests run in milliseconds; run them constantly.

WHEN IT MATTERS cargo test lists each test by name with ok.

© 2026 Tertiary Infotech Academy Pte Ltd

Unit tests live beside the code / failure



Symptom present: fix

Predict the expected value from the specification before running, never copy actual output.

Contract holds: evidence

cargo test lists each test by name with ok.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / ARCH

Integration tests check components together

Read the code, then the contract: what must always be true?

```
tests/integration.rs      # separate crate, public API only

use activity11::{parse_lines, summarise, Summary};
const SAMPLE: &str = include_str!("../../data/stock.csv");

#[test]
fn end_to_end_summary() {
    assert_eq!(summarise(SAMPLE, 5), Ok(Summary {
        lines: 4, total_cents: 52_655,
        low_stock: vec!["ABC-0003".into()] }));
}
```

tests/ folder

Each file is compiled like an external user of the crate.

Fixtures

include_str! embeds a data file at compile time.

End to end

Parse, calculate and report are checked together.

Public API

If a test needs a private item, the API may be incomplete.

© 2026 Tertiary Infotech Academy Pte Ltd

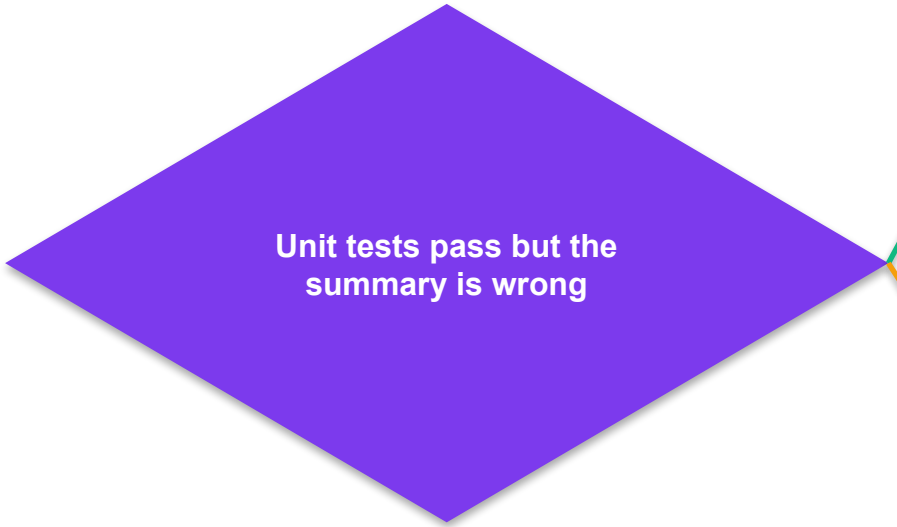
Integration tests check components together / contract

Contract field	Exact rule
Unit	One function, fast, may use private items.
Integration	Several components through the public API.
Process	Run the real binary (Activity 12).
Data	Fixture files live in data/ and are versioned.

WHEN IT MATTERS Activity 11: the end-to-end test fixes the expected total at 52,655 cents.

© 2026 Tertiary Infotech Academy Pte Ltd

Integration tests check components together / failure



Symptom present: fix

Components disagree; add an integration test with a known fixture total.

Contract holds: evidence

Activity 11: the end-to-end test fixes the expected total at 52,655 cents.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Custom errors and the ? operator

Read the code, then the contract: what must always be true?

```
#[derive(Debug, PartialEq)]
pub enum ParseError {
    FieldCount(usize),
    EmptySku,
    BadNumber { field: &'static str, value: String },
}
impl fmt::Display for ParseError { .. }
impl std::error::Error for ParseError {}

let qty = number("qty", fields[1])?;   // early return on Err
```

Error enum

Each failure is a named, testable variant.

Display

Human-readable message for logs and operators.

Error trait

Lets the error work with Box<dyn Error> and ?.

?

Stops at the first bad field and returns its error.

© 2026 Tertiary Infotech Academy Pte Ltd

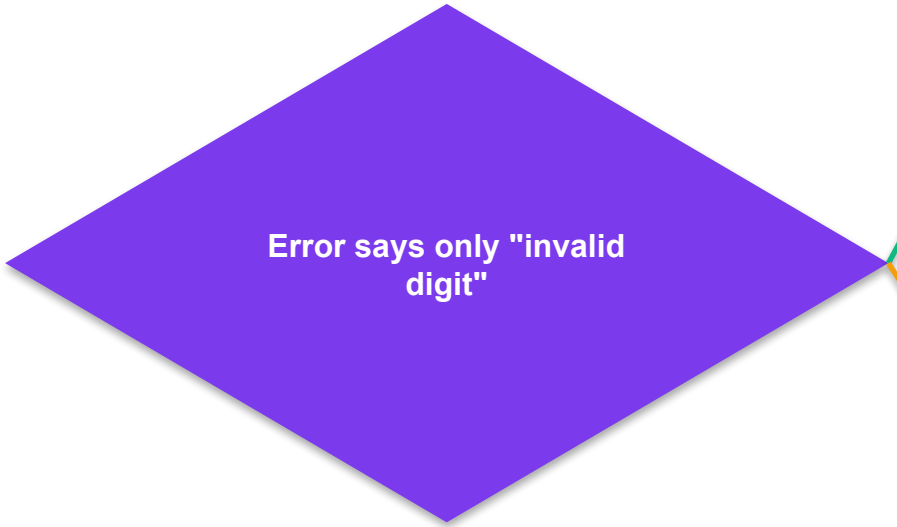
Custom errors and the ? operator / contract

Contract field	Exact rule
Recoverable	Bad input is Result, not panic.
Context	Include the field and the offending value.
Aggregation	parse_all keeps good lines and numbered errors.
Tests	assert_eq! on the exact error variant and message.

WHEN IT MATTERS Activity 10 reports errors on lines 3, 4 and 6 with readable messages.

© 2026 Tertiary Infotech Academy Pte Ltd

Custom errors and the ? operator / failure



Symptom present: fix

Map low-level errors into your enum with field name and value via map_err.

Contract holds: evidence

Activity 10 reports errors on lines 3, 4 and 6 with readable messages.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / EVIDENCE

Read a failing test like a detective

Read the code, then the contract: what must always be true?

```
---- total_includes_the_first_line stdout ----
thread 'total_includes_the_first_line' panicked at
tests/integration.rs:13:5:
assertion `left == right` failed
  left: Some(16695)
 right: Some(52655)

52655 - 16695 = 35960 = 40 x 899  -> first line missing
```

Test name

Tells you which rule is broken.

left / right

Actual value versus expected value.

Arithmetic

The difference often identifies the missing or extra data.

Location

File and line of the failing assertion.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / EXACT INTERFACE

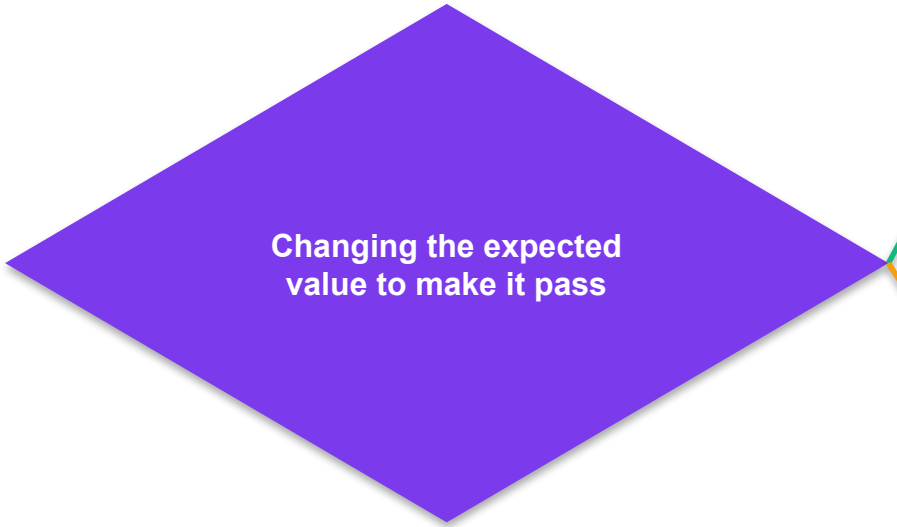
Read a failing test like a detective / contract

Contract field	Exact rule
Reproduce	Run one test: cargo test total_includes
Backtrace	RUST_BACKTRACE=1 cargo test shows the call path.
Isolate	Check each component with its own test.
Fix	Change the code, never the expected value.

WHEN IT MATTERS After the fix, the same test passes and stays as a regression test.

© 2026 Tertiary Infotech Academy Pte Ltd

Read a failing test like a detective / failure



Symptom present: fix

Forbidden: the expected value comes from the specification; find the defect instead.

Contract holds: evidence

After the fix, the same test passes and stays as a regression test.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / EVIDENCE

Debug with dbg!, focused tests and AI

Read the code, then the contract: what must always be true?

```
let total = dbg!(total_cents(&lines));   // prints file:line + value

cargo test low_stock -- --nocapture      # one test, show output
RUST_BACKTRACE=1 cargo test

Prompt: "Here is the failing test and total_cents().
Explain why the actual value is 16695, point to the line,
and propose the smallest fix. Do not change the test."
```

dbg!

Prints the expression, its value and where it ran; returns the value.

Filter

cargo test <name> runs only matching tests.

--nocapture

Shows println!/dbg! output from passing tests too.

AI

Ask for a diagnosis first, then a minimal patch.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / EXACT INTERFACE

Debug with dbg!, focused tests and AI / contract

Contract field	Exact rule
Hypothesis	State what you think is wrong before changing code.
One change	Change one thing, rerun, observe.
Cleanup	Remove dbg! before committing; clippy can flag it.
Record	Symptom, cause, fix and test in the review record.

WHEN IT MATTERS Activity 11 ends with 6 passed and a written defect record.

© 2026 Tertiary Infotech Academy Pte Ltd

Debug with dbg!, focused tests and AI / failure



Symptom present: fix

Run the whole suite after every change, not only the failing test.

Contract holds: evidence

Activity 11 ends with 6 passed and a written defect record.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Interoperability: arguments, files, exit codes

Read the code, then the contract: what must always be true?

```
$ stockpilot data/stock.csv 13
ITEMS 4
TOTAL $526.55
LOW STOCK ABC-0002, ABC-0003, ABC-0004
$ echo $?        # 0

$ stockpilot            # usage error -> stderr, exit 2
$ stockpilot nope.csv   # cannot read -> stderr, exit 1
```

Arguments

std::env::args(); validate count and types.

Files

std::fs::read_to_string returns Result.

stdout/stderr

Data to stdout, diagnostics to stderr.

ExitCode

main returns ExitCode so scripts can react.

© 2026 Tertiary Infotech Academy Pte Ltd

Interoperability: arguments, files, exit codes / contract

Contract field	Exact rule
0	Success: report printed.
1	Data problem: unreadable file or malformed row.
2	Usage problem: wrong arguments.
Tests	Process tests run the binary via CARGO_BIN_EXE_<name>.

WHEN IT MATTERS Activity 12 process tests check stdout, stderr and exit codes 0, 1 and 2.

© 2026 Tertiary Infotech Academy Pte Ltd

Interoperability: arguments, files, exit codes / failure



Symptom present: fix

Return a non-zero ExitCode and print the error to stderr.

Contract holds: evidence

Activity 12 process tests check stdout, stderr and exit codes 0, 1 and 2.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Serde: typed data at the boundary

Read the code, then the contract: what must always be true?

```rust
#[derive(Serialize, Deserialize, Debug, PartialEq)]
pub struct Item {
    pub sku: String,
    pub name: String,
    pub qty: u32,          // -1 in JSON -> error
    pub unit_cents: u64,
}

let items: Vec<Item> = serde_json::from_str(json)?;
let text = serde_json::to_string_pretty(&items)?;
```

Derive

Serialize/Deserialize are generated from the struct.

Validation

Types reject wrong shapes before business logic runs.

Round trip

Serialise then parse must give equal values.

Errors

serde_json::Error reports line, column and cause.

© 2026 Tertiary Infotech Academy Pte Ltd

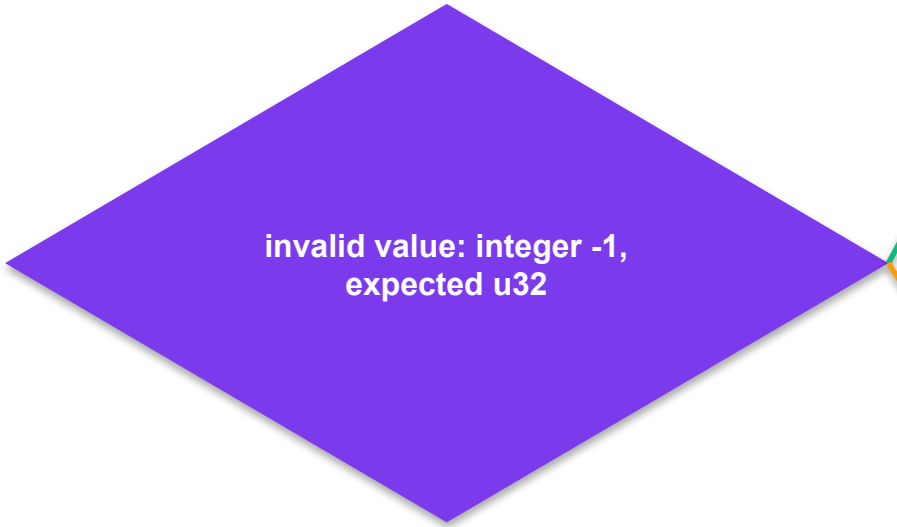
Serde: typed data at the boundary / contract

Contract field	Exact rule
Missing field	Error unless #[serde(default)] is chosen.
Unknown field	Ignored unless #[serde(deny_unknown_fields)].
Numbers	Out-of-range or negative values for u32 are errors.
Contract	The struct is the documented JSON schema.

WHEN IT MATTERS Activity 6 rejects a missing field and a negative quantity.

© 2026 Tertiary Infotech Academy Pte Ltd

Serde: typed data at the boundary / failure



Symptom present: fix

Correct the source data; do not widen the type to accept impossible values.

Contract holds: evidence

Activity 6 rejects a missing field and a negative quantity.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / 30 MIN HANDS-ON

Parse CSV records with Result and custom errors

ACTIVITY 10

Independent practice; full steps in the activity README and Learner Guide

Stock files arrive with typos. Instead of crashing, StockPilot must report exactly which line is wrong and why, while still importing every valid line.

YOU'LL BUILD

labs/activity-10-error-handling: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 5 passed; data/stock.csv yields 2 records and errors on lines 3, 4 and 6.

© 2026 Tertiary Infotech Academy Pte Ltd

Parse CSV records with Result and custom errors / learn it first

samples/01_result_and_question_mark.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: Result and the ? operator.
use std::num::ParseIntError;

fn parse_qty(text: &str) -> Result<u32, ParseIntError> {
    text.trim().parse::<u32>()
}

fn total_qty(a: &str, b: &str) -> Result<u32, ParseIntError> {
    let first = parse_qty(a)?; // returns early on Err
    let second = parse_qty(b)?;
    Ok(first + second)
}

fn main() {
    println!("{:?}", total_qty("4", " 6 "));
    match total_qty("4", "six") {
// ... full script: samples/01_result_and_question_mark.rs
```

Run the sample

rustc --edition 2021 01_result_and_question_mark.rs && ./01_result_and_question_mark

Explain it simply

Ask the AI to explain Result, the ? operator, custom error enums, Display and Box<dyn Error>.

Quiz me

5 questions, one at a time, including a predict-the-output question.

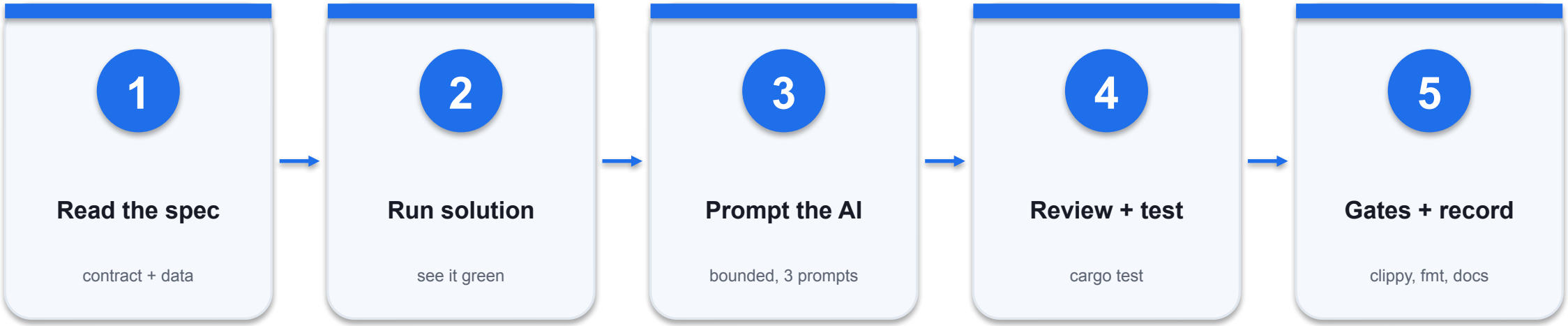
Compare with what I know

Rust vs Python / JavaScript for error handling.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 10 / WORKFLOW

Parse CSV records with Result and custom errors / workflow



EXACT CONTRACT

parse_record(&str) -> Result<Record, ParseError> with ParseError::{FieldCount(n), EmptySku, BadNumber{field, value}} and a Display message for each. parse_all(&str) skips blank lines and the header, returning valid records and (1-based line number, error) pairs.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 10 / CODE

Parse CSV records with Result and custom errors / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Parses `sku,qty,unit_cents`; surrounding spaces are ignored.
pub fn parse_record(line: &str) -> Result<Record, ParseError> {
    let fields: Vec<&str> = line.split(',').map(str::trim).collect();
    if fields.len() != 3 {
        return Err(ParseError::FieldCount(fields.len()));
    }
    if fields[0].is_empty() {
        return Err(ParseError::EmptySku);
    }
    Ok(Record {
        sku: fields[0].to_string(),
        qty: number("qty", fields[1])?,
        unit_cents: number("unit_cents", fields[2])?,
    })
}
```

Result

Every caller must decide what to do with Ok and Err.

Custom error

An enum names each failure so tests can match on it exactly.

Display

Separate, human-readable messages for operators and logs.

?

Returns the error early, converting it where From allows.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 10 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Parse CSV records with Result and custom errors / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: parse_record(&str) -> Result<Record, ParseError> with ParseError::{FieldCount(n), EmptySku, BadNumber{field, value}} and a Display message for each. parse_all(&str) skips blank lines and the header, returning valid records and (1-based line number, error) pairs. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Parse CSV records with Result and custom errors" in starter/src so that the supplied tests pass. Return Result everywhere; no unwrap(), expect() or panic! in library code. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 10 / EVIDENCE

Parse CSV records with Result and custom errors / acceptance

✅ Expected result

cargo test reports 5 passed; data/stock.csv yields 2 records and errors on lines 3, 4 and 6. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / 35 MIN HANDS-ON

Debug a failing integration with tests

ACTIVITY 11

Independent practice; full steps in the activity README and Learner Guide

The nightly stock summary is wrong: the total is too low and a correctly stocked item is flagged. The starter crate contains two real defects. You reproduce them with tests, diagnose them with the help of an AI assistant, fix them and keep regression tests.

YOU'LL BUILD

labs/activity-11-debug-integration: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

Starter: 4 of 6 tests fail. After both fixes cargo test reports 6 passed.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 11 / LEARN RUST

Debug a failing integration with tests / learn it first

samples/01_dbg_and_assert.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: dbg! and assert_eq! as debugging tools.
fn total(values: &[u64]) -> u64 {
    values.iter().sum()
}

fn main() {
    let values = [35_960, 12_495, 0, 4_200];
    let result = dbg!(total(&values)); // prints file:line and value to stderr
    assert_eq!(result, 52_655, "total must include every line");
    let skipped: u64 = values.iter().skip(1).sum();
    println!("correct total {result}, total without the first line
{skipped}");
    println!("difference {} = the first line", result - skipped);
}
```

Run the sample

rustc --edition 2021 01_dbg_and_assert.rs &&
./01_dbg_and_assert

Explain it simply

Ask the AI to explain assertions, dbg!, unit tests with #[test], and reading a failing test.

Quiz me

5 questions, one at a time, including a predict-the-output question.

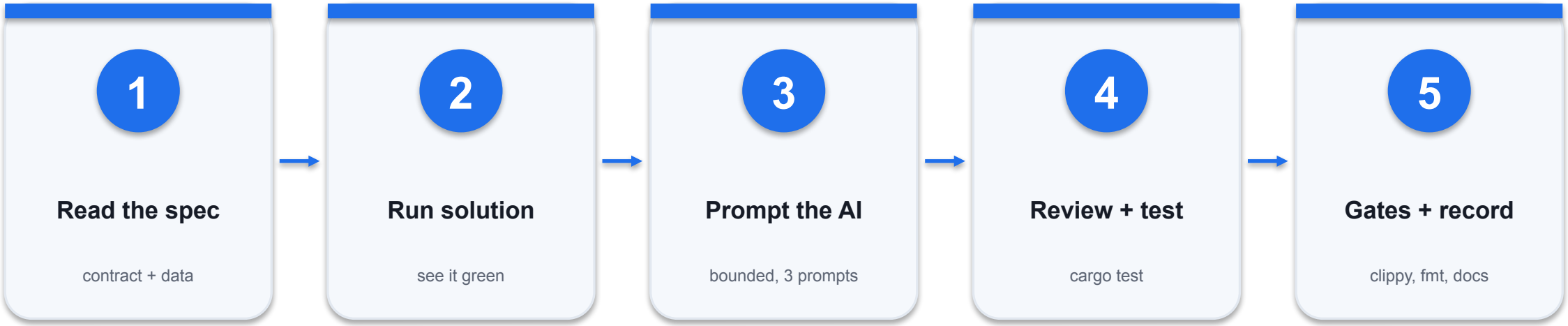
Compare with what I know

Rust vs Python / JavaScript for testing and debugging.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 11 / WORKFLOW

Debug a failing integration with tests / workflow



EXACT CONTRACT

summarise(csv, threshold) -> Result<Summary, String> where Summary { lines, total_cents, low_stock }. Every line counts towards the total; low stock is strictly below the threshold, sorted by SKU; malformed lines are Err("line N: ...").

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 11 / CODE

Debug a failing integration with tests / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Total value of every line, or `None` on overflow.
pub fn total_cents(lines: &[Line]) -> Option<u64> {
    lines.iter().try_fold(0u64, |total, line| {
        total.checked_add(line.unit_cents.checked_mul(u64::from(line.qty))?)
    })
}
```

Reproduce

A failing test pins the defect with exact input and expected output.

Diagnose

Compare expected and actual values; 16,695 vs 52,655 means a line is missing.

Fix minimally

Change the defect, never the expected values in the test.

Regress

Keep the test so the defect cannot silently return.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 11 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Debug a failing integration with tests / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: summarise(csv, threshold) -> Result<Summary, String> where Summary { lines, total_cents, low_stock }. Every line counts towards the total; low stock is strictly below the threshold, sorted by SKU; malformed lines are Err("line N: ..."). Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Debug a failing integration with tests" in starter/src so that the supplied tests pass. Give the AI the failing test output and the function only; ask for a diagnosis before any code change. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 11 / EVIDENCE

Debug a failing integration with tests / acceptance

✅ Expected result

Starter: 4 of 6 tests fail. After both fixes cargo test reports 6 passed. Starter before you begin: Four of the six tests fail because the starter contains two real defects; the other two pass.

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K4/A4 / 30 MIN HANDS-ON

Integrate components into a tested command-line tool

ACTIVITY 12

Independent practice; full steps in the activity README and Learner Guide

Operations staff will run StockPilot from a terminal and from scheduled scripts. You connect the library to a binary that reads a file, prints a report and returns exit codes that other programs can rely on.

YOU'LL BUILD

labs/activity-12-cli-integration: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 7 passed; cargo run -- ../data/stock.csv prints ITEMS 4, TOTAL $526.55, LOW STOCK ABC-0003.

© 2026 Tertiary Infotech Academy Pte Ltd

Integrate components into a tested command-line tool / learn it first

samples/01_read_file_and_exit_code.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: read a file named on the command line; exit code 1 if it fails.
use std::process::ExitCode;

fn main() -> ExitCode {
    let path = std::env::args()
        .nth(1)
        .unwrap_or_else(|| "missing.csv".to_string());
    match std::fs::read_to_string(&path) {
        Ok(text) => {
            println!("{path}: {} line(s)", text.lines().count());
            ExitCode::SUCCESS
        }
        Err(error) => {
            eprintln!("error: cannot read {path}: {error}");
            ExitCode::from(1)
        }
// ... full script: samples/01_read_file_and_exit_code.rs
```

Run the sample

rustc --edition 2021 01_read_file_and_exit_code.rs &&
./01_read_file_and_exit_code ../data/stock.csv

Explain it simply

Ask the AI to explain reading files, command-line arguments, stdout versus stderr and process exit codes.

Quiz me

5 questions, one at a time, including a predict-the-output question.

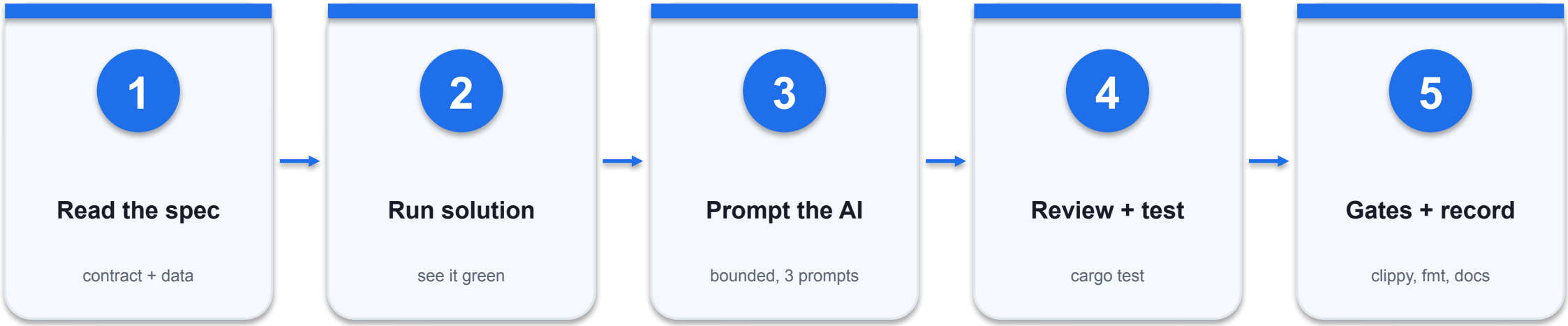
Compare with what I know

Rust vs Python / JavaScript for building command-line tools.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 12 / WORKFLOW

Integrate components into a tested command-line tool / workflow



EXACT CONTRACT

run(csv, threshold) -> Result<String, String> returns "ITEMS n\nTOTAL $D.CC\nLOW STOCK a, b | none". The binary takes <file> [threshold=5]; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 12 / CODE

Integrate components into a tested command-line tool / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```
pub fn run(csv: &str, threshold: u32) -> Result<String, String> {
    let mut count = 0usize;
    let mut total = 0u64;
    let mut low = Vec::new();
    for (index, raw) in csv.lines().enumerate() {
        let raw = raw.trim();
        if raw.is_empty() || raw.starts_with("sku,") {
            continue;
        }
        let fields: Vec<&str> = raw.split(',').map(str::trim).collect();
        let [sku, qty, unit] = fields.as_slice() else {
            return Err(format!("line {}: expected 3 fields", index + 1));
        };
        let qty: u32 = qty
            .parse()
            .map_err(|_| format!("line {}: bad qty", index + 1))?;
    // ... full code: solution/src/lib.rs
```

Thin main

main handles arguments, files and exit codes; the library holds the logic.

Exit codes

0 success, 1 bad data, 2 usage: a contract for scripts and schedulers.

CARGO_BIN_EXE

Integration tests run the real compiled binary as a separate process.

stdout vs stderr

Reports go to stdout; errors go to stderr so pipes stay clean.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 12 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Integrate components into a tested command-line tool / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: run(csv, threshold) -> Result<String, String> returns "ITEMS n\nTOTAL $D.CC\nLOW STOCK a, b | none". The binary takes <file> [threshold=5]; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Integrate components into a tested command-line tool" in starter/src so that the supplied tests pass. Keep main thin; put every rule in lib.rs where the library tests can reach it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 12 / EVIDENCE

Integrate components into a tested command-line tool / acceptance

✅ Expected result

cargo test reports 7 passed; cargo run -- ../data/stock.csv prints ITEMS 4, TOTAL $526.55, LOW STOCK ABC-0003. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

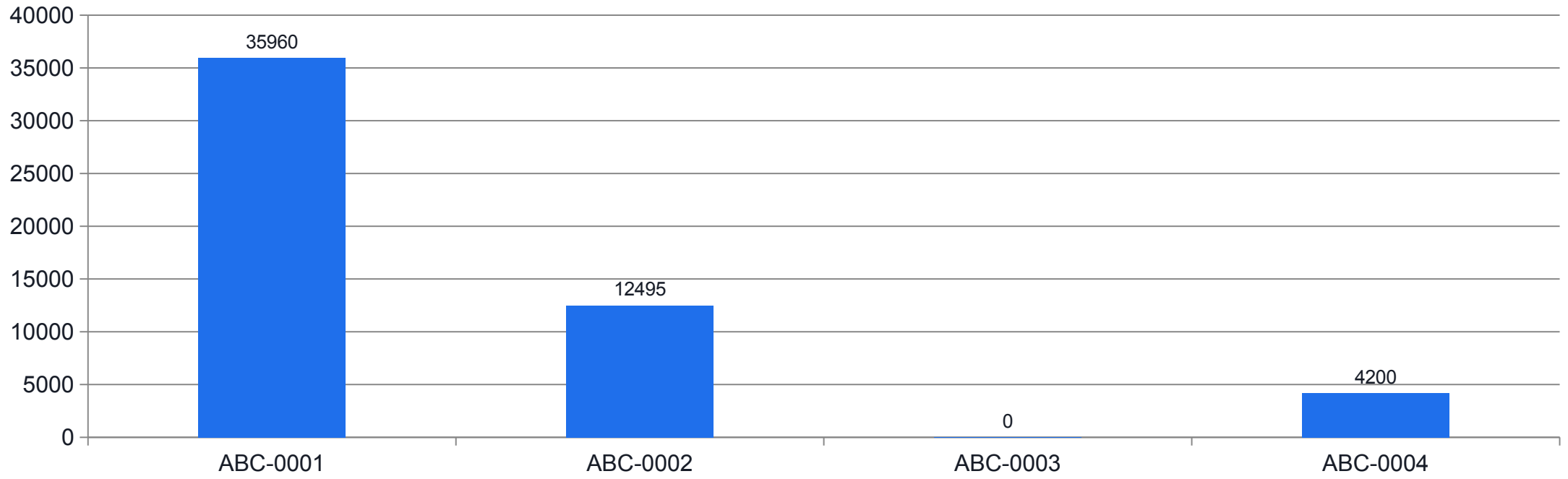
Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

Activity 11 fixture: stock value by SKU (cents)



WHAT THE DATA SHOWS

Correct total 52,655. The defective starter reported 16,695: exactly the total without ABC-0001 (35,960). The arithmetic points straight at the skipped first line.

© 2026 Tertiary Infotech Academy Pte Ltd

5

TOPIC 5 / K5/A5

AI-Assisted Rust Software Design and Documentation

Activities 13-14 / concepts, contracts, failure modes and evidence

© 2026 Tertiary Infotech Academy Pte Ltd

Doc comments become documentation

Read the code, then the contract: what must always be true?

```
//! Reorder-point calculations for StockPilot.   (crate docs)
#![deny(missing_docs)]

/// Calculates the reorder point in units.
///
/// `point = daily_usage x lead_days + safety_stock` (R1).
/// Returns `None` instead of wrapping on overflow (R2).
pub fn reorder_point(daily_usage: u32, lead_days: u32,
                     safety_stock: u32) -> Option<u32>

cargo doc --no-deps --open
```

//!

Documents the enclosing item: the crate or module.

///

Documents the next item: function, struct, field.

Markdown

Headings, lists, code and links render in cargo doc.

deny(missing_docs)

An undocumented public item fails the build.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EXACT INTERFACE

Doc comments become documentation / contract

Contract field	Exact rule
Summary line	One sentence saying what it does.
Units and ranges	State units (cents, days) and limits.
Failure	Say what None/Err means, or add # Panics / # Errors.
Trace	Reference requirement IDs such as R1.

WHEN IT MATTERS cargo doc builds and clippy -D warnings reports no missing docs.

© 2026 Tertiary Infotech Academy Pte Ltd

Doc comments become documentation / failure



Symptom present: fix

Add a /// summary with units, return meaning and an example.

Contract holds: evidence

cargo doc builds and clippy -D warnings reports no missing docs.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EVIDENCE

Doctests keep examples honest

Read the code, then the contract: what must always be true?

```
/// # Examples
///
/// ```
/// use activity13::reorder_point;
/// assert_eq!(reorder_point(12, 5, 20), Some(80));
/// ```
pub fn reorder_point(..) -> Option<u32> { .. }

$ cargo test
   Doc-tests activity13
test src/lib.rs - reorder_point (line 12) ... ok
```

Doctest

A code block in docs is compiled and run by cargo test.

Honesty

If behaviour changes, the stale example fails the build.

Users first

Examples show how to call the API correctly.

AI drafts

AI-written examples must pass as doctests before acceptance.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EXACT INTERFACE

Doctests keep examples honest / contract

Contract field	Exact rule
Location	Under a # Examples heading in /// comments.
Imports	Doctests use the crate like an external user.
Hidden lines	Lines starting with # are compiled but not shown.
Evidence	The Doc-tests section of cargo test output.

WHEN IT MATTERS Activity 13: 3 unit tests and 3 doctests pass.

© 2026 Tertiary Infotech Academy Pte Ltd

Doctests keep examples honest / failure



Symptom present: fix

Update the example to the new, specified behaviour, or fix the regression it caught.

Contract holds: evidence

Activity 13: 3 unit tests and 3 doctests pass.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / MODEL

Trace requirements to code and tests

Read the code, then the contract: what must always be true?

```
ID  Requirement              Test (tests/acceptance.rs)
R1  point = usage*lead+safe  r1_r3_below_the_point_...
R2  above point: no order    r2_above_the_point_...
R3  order up to 7-day target r2_exactly_at_the_point_...
R4  overflow item skipped    r4_overflow_is_skipped_...

Code: reorder_suggestions() in src/lib.rs
R1  checked_mul(..)?.checked_add(..)?
R2  if item.on_hand > point { return None; }
R3  point + cover - on_hand, kept only if > 0
R4  ? after every checked_* call
```

Trace matrix

Each user requirement links to code and a named test.

Coverage gap

An empty cell is a missing test or feature.

Change impact

Changing R3 shows exactly which code and tests to update.

Audit

Reviewers can verify claims without reading all code.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EXACT INTERFACE

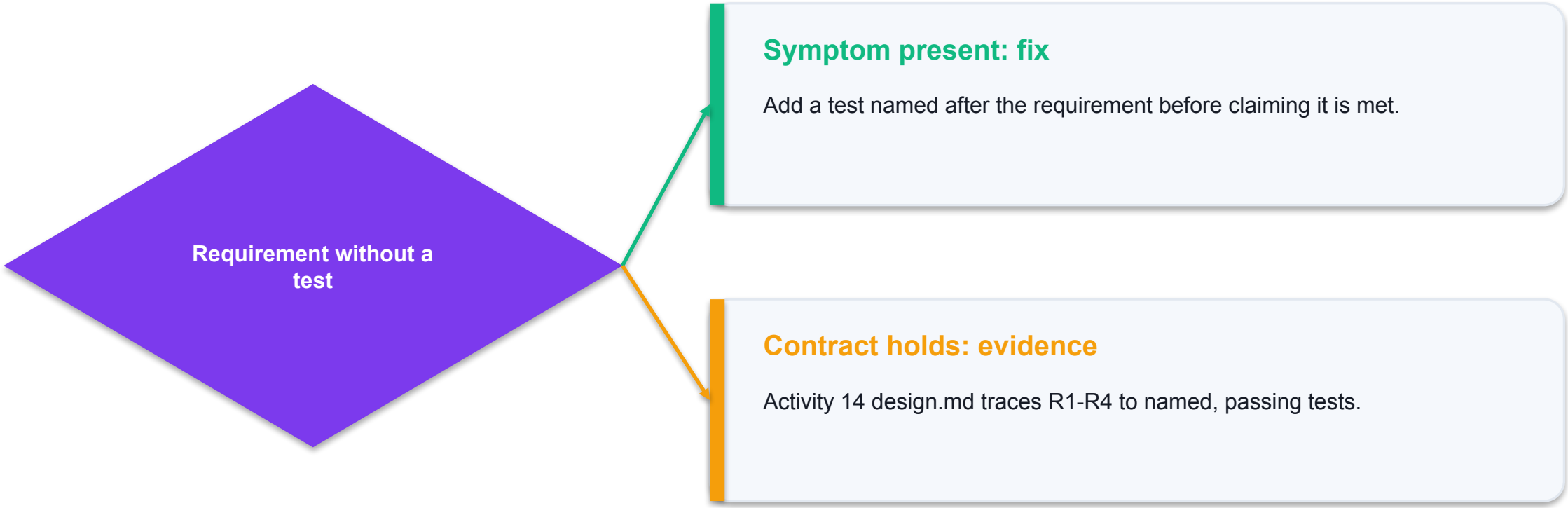
Trace requirements to code and tests / contract

Contract field	Exact rule
Requirement ID	Stable identifiers R1..Rn used in docs, tests and commits.
Code	Function or line that implements it.
Test	Exact test name; the result must be ok.
Status	Passed, failed or not yet run: never assumed.

WHEN IT MATTERS Activity 14 design.md traces R1-R4 to named, passing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

Trace requirements to code and tests / failure



Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Architecture and module maps

Read the code, then the contract: what must always be true?



Components

One box per module with a single responsibility.

Data flow

Arrows labelled with the Rust types that cross them.

Boundaries

I/O at the edges; pure logic in the middle.

Mermaid

Diagrams as text in Markdown, versioned with the code.

© 2026 Tertiary Infotech Academy Pte Ltd

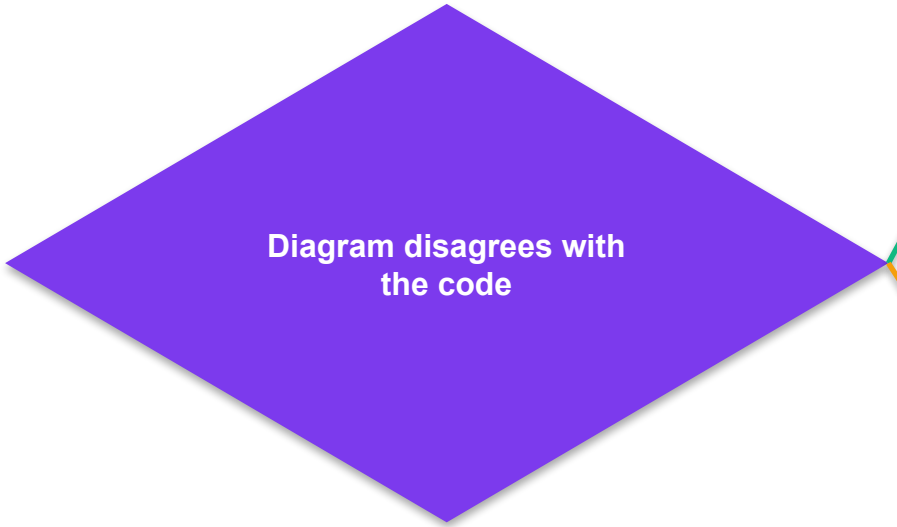
Architecture and module maps / contract

Contract field	Exact rule
Module	Name, responsibility, public functions.
Interface	Input type, output type, error type.
External	Files, arguments, exit codes, crates.
Owner	Who maintains it and where its tests live.

WHEN IT MATTERS Every box in design.md exists as a module or function in the crate.

© 2026 Tertiary Infotech Academy Pte Ltd

Architecture and module maps / failure



Symptom present: fix

Generate the module list from lib.rs and update the diagram in the same change.

Contract holds: evidence

Every box in design.md exists as a module or function in the crate.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / MODEL

Record design decisions

Read the code, then the contract: what must always be true?

```
Decision:  Skip items whose arithmetic overflows (R4).
Context:   Planning data can contain bad usage values.
Options:   1. panic  2. saturate at u32::MAX  3. skip item
Chosen:    3 - skip; wrong orders are worse than none.
Consequences:
  + no crash, no absurd order quantities
  - skipped items must be visible in a report (future)
Evidence:  test r4_overflow_is_skipped_without_panic
```

Decision

What was decided, in one sentence.

Options

Alternatives considered, so reviewers see the trade-off.

Consequences

Benefits and costs, including follow-up work.

Evidence

The test that demonstrates the decision.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EXACT INTERFACE

Record design decisions / contract

Contract field	Exact rule
When	Any choice a maintainer would question later.
Where	docs/design.md next to the code.
AI role	AI may list options; you choose and justify.
Update	Supersede, do not delete, old decisions.

WHEN IT MATTERS design.md contains at least one complete decision record.

© 2026 Tertiary Infotech Academy Pte Ltd

Record design decisions / failure



Symptom present: fix

Add a decision record with context, options and a linked test.

Contract holds: evidence

design.md contains at least one complete decision record.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EVIDENCE

AI-assisted documentation, human-verified

Read the code, then the contract: what must always be true?

```
Prompt: "Draft rustdoc for reorder_suggestions using ONLY
this code and R1-R4. Include units, a # Examples doctest
and a requirement list. Mark anything you are unsure of."

Verify:
  cargo test          # doctest compiles and passes
  cargo doc --no-deps # renders
  compare every claim with code and tests
```

Grounding

Give the AI the code and requirements, nothing else.

Hallucination

AI may describe behaviour the code does not have.

Doctests

Executable examples catch false claims automatically.

Ownership

You sign off the documentation, not the AI.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EXACT INTERFACE

AI-assisted documentation, human-verified / contract

Contract field	Exact rule
Accept	Claims that match code and a passing test.
Correct	Wrong units, ranges or missing failure cases.
Reject	Invented functions, options or performance claims.
Record	Section, what was verified, what was changed.

WHEN IT MATTERS The AI review table in design.md lists verified and corrected sections.

© 2026 Tertiary Infotech Academy Pte Ltd

AI-assisted documentation, human-verified / failure



Symptom present: fix

Delete the claim or implement and test it; never leave it unverified.

Contract holds: evidence

The AI review table in design.md lists verified and corrected sections.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

Hand over: README and maintenance notes

Read the code, then the contract: what must always be true?

```
README.md
  What it does           (one paragraph)
  Build and test         cargo test / clippy / fmt
  Usage                  stockpilot <file> [threshold]
  Requirements trace     link to docs/design.md
  Known limits           e.g. skipped overflow items
  Change procedure       update code + docs + tests together
```

Audience

Written for the maintainer who joins next month.

Commands

Exact commands that prove the system still works.

Limits

Honest list of what is not handled yet.

Procedure

How to make a change safely.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / EXACT INTERFACE

Hand over: README and maintenance notes / contract

Contract field	Exact rule
Build	Toolchain version and commands.
Interfaces	Inputs, outputs, exit codes and data formats.
Quality gates	fmt, clippy -D warnings, test, doc.
Contacts	Owner and where decisions are recorded.

WHEN IT MATTERS A classmate can build, test and explain the capstone from its README and design.md.

© 2026 Tertiary Infotech Academy Pte Ltd

Hand over: README and maintenance notes / failure



Symptom present: fix

Write down commands, limits and decisions until a newcomer can run and change it alone.

Contract holds: evidence

A classmate can build, test and explain the capstone from its README and design.md.

Compiler errors, failing tests and clippy warnings are evidence: read them, then act.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 / 30 MIN HANDS-ON

Document an API with rustdoc and doctests

ACTIVITY 13

Independent practice; full steps in the activity README and Learner Guide

The maintenance team will own the reorder calculation. The starter code works but has no documentation. You write rustdoc comments whose examples are compiled and run as tests, and trace them to requirements.

YOU'LL BUILD

labs/activity-13-rustdoc-doctests: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test runs 3 tests and 3 doctests; cargo doc --no-deps builds; clippy -D warnings is clean.

© 2026 Tertiary Infotech Academy Pte Ltd

Document an API with rustdoc and doctests / learn it first

samples/01_doc_comments.rs: predict the output, run it, then use the Learn Rust prompts

```rust
//! Sample library: document an API so rustdoc can render it and test it.

/// Days of usage kept as safety stock.
pub const SAFETY_DAYS: u32 = 2;

/// Returns the safety stock in units for a daily usage.
///
/// # Examples
///
/// ```
/// assert_eq!(docsample::safety_stock(12), Some(24));
/// assert_eq!(docsample::safety_stock(u32::MAX), None);
/// ```
pub fn safety_stock(daily_usage: u32) -> Option<u32> {
    daily_usage.checked_mul(SAFETY_DAYS)
}
```

Run the sample

rustc --crate-type lib, then rustdoc --test -L . (exact commands: samples/README.md)

Explain it simply

Ask the AI to explain doc comments (/// and //!), # Examples sections, doctests and rustdoc.

Quiz me

5 questions, one at a time, including a predict-the-output question.

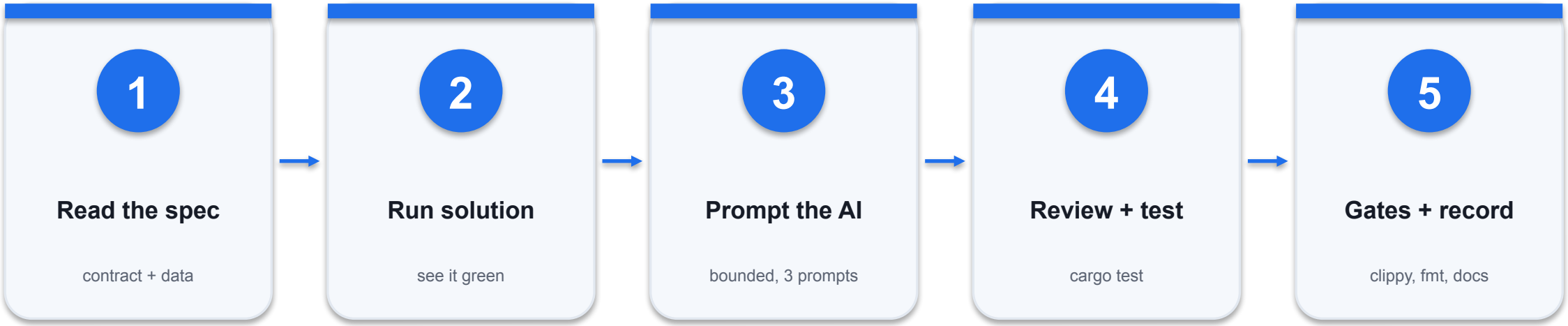
Compare with what I know

Rust vs Python / JavaScript for writing documentation.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 13 / WORKFLOW

Document an API with rustdoc and doctests / workflow



EXACT CONTRACT

reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock (R1), None on overflow (R2). needs_reorder(on_hand, point) is true at or below the point (R3). Every public item is documented; cargo clippy with -D warnings enforces missing_docs.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 13 / CODE

Document an API with rustdoc and doctests / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
/// Calculates the reorder point in units.
///
/// `reorder point = daily_usage x lead_days + safety_stock` (R1).
///
/// # Examples
///
/// ```
/// ... (full docs in the source file)
pub fn reorder_point(
    daily_usage: u32,
    lead_days: u32,
    safety_stock: u32,
) -> Option<u32> {
    daily_usage
        .checked_mul(lead_days)?
        .checked_add(safety_stock)
}
```

///

Outer doc comments document the next item; //! documents the crate.

Examples

Code blocks in docs are compiled and run by cargo test as doctests.

missing_docs

deny(missing_docs) turns an undocumented public item into a build error.

Traceability

R1-R3 in the docs link each behaviour to a user requirement and a test.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 13 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Document an API with rustdoc and doctests / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock (R1), None on overflow (R2). needs_reorder(on_hand, point) is true at or below the point (R3). Every public item is documented; cargo clippy with -D warnings enforces missing_docs. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Document an API with rustdoc and doctests" in starter/src so that the supplied tests pass. Ask the AI to draft docs from the code and requirements only; verify every example by running cargo test. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 13 / EVIDENCE

Document an API with rustdoc and doctests / acceptance

✅ Expected result

cargo test runs 3 tests and 3 doctests; cargo doc --no-deps builds; clippy -D warnings is clean. Starter before you begin: cargo test passes because the logic already works, but cargo clippy --all-targets -- -D warnings fails with "missing documentation" errors.

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

K5/A5 K4/A4 / 30 MIN HANDS-ON

Design and document a reorder feature with AI

ACTIVITY 14

Independent practice; full steps in the activity README and Learner Guide

Capstone: purchasing wants weekly reorder suggestions. You agree requirements R1-R4, implement the feature with an AI assistant, and hand over a design record that traces every requirement to code and tests.

YOU'LL BUILD

labs/activity-14-design-capstone: complete starter/ against the tests

TOOLCHAIN

Rust stable / Cargo / VS Code + rust-analyzer / AI assistant

DONE WHEN

cargo test reports 6 passed and 1 doctest; docs/design.md traces R1-R4 to named tests.

© 2026 Tertiary Infotech Academy Pte Ltd

Design and document a reorder feature with AI / learn it first

samples/01_design_with_types.rs: predict the output, run it, then use the Learn Rust prompts

```rust
// Sample: newtypes make invalid values hard to create.
struct Units(u32);

struct Sku(String);

impl Sku {
    fn parse(text: &str) -> Option<Sku> {
        let valid = text.len() == 8 && text.as_bytes()[3] == b'-';
        valid.then(|| Sku(text.to_string()))
    }
}

fn order(sku: &Sku, qty: Units) -> String {
    format!("order {} x {}", qty.0, sku.0)
}

// ... full script: samples/01_design_with_types.rs
```

Run the sample

rustc --edition 2021 01_design_with_types.rs && ./01_design_with_types

Explain it simply

Ask the AI to explain designing with types (newtypes), requirement-named tests and design trade-offs.

Quiz me

5 questions, one at a time, including a predict-the-output question.

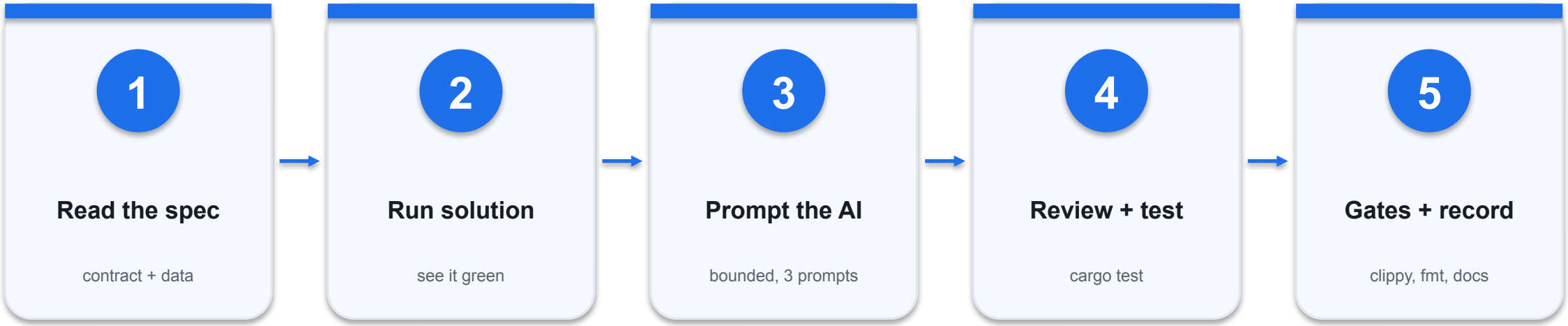
Compare with what I know

Rust vs Python / JavaScript for software design and traceability.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 14 / WORKFLOW

Design and document a reorder feature with AI / workflow



EXACT CONTRACT

reorder_suggestions(&[StockItem]) -> Vec<Suggestion>. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 order_qty = point + 7 days usage - on_hand, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 14 / CODE

Design and document a reorder feature with AI / Rust

Reference solution: solution/src/lib.rs (formatted, tested, clippy-clean)

```rust
pub fn reorder_suggestions(items: &[StockItem]) -> Vec<Suggestion> {
    let mut suggestions: Vec<Suggestion> = items
        .iter()
        .filter_map(|item| {
            let point = item
                .daily_usage
                .checked_mul(item.lead_days)?
                .checked_add(item.safety_stock)?;
            if item.on_hand > point {
                return None;
            }
            let cover = item.daily_usage.checked_mul(REVIEW_DAYS)?;
            let order_qty = point.checked_add(cover)? - item.on_hand;
            (order_qty > 0).then(|| Suggestion {
                sku: item.sku.clone(),
                order_qty,
    // ... full code: solution/src/lib.rs
```

Requirements first

R1-R4 are agreed before prompting, so AI output has a target.

filter_map

One pass: skip items (None) or produce a Suggestion (Some).

No underflow

on_hand <= point <= target, so the subtraction cannot underflow.

Design record

docs/design.md traces R1-R4 to code, tests and review decisions.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 14 / PROMPTS (ALSO IN PROMPTS.md AND THE PDF)

Design and document a reorder feature with AI / vibe-coding prompts

Plan before code

- Act as a senior Rust reviewer. Specification: reorder_suggestions(&[StockItem]) -> Vec<Suggestion>. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 order_qty = point + 7 days usage - on_hand, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

- Implement "Design and document a reorder feature with AI" in starter/src so that the supplied tests pass. Give the AI R1-R4 and the public types; require a trace table in its answer and verify each row against the tests. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

- Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

© 2026 Tertiary Infotech Academy Pte Ltd

ACTIVITY 14 / EVIDENCE

Design and document a reorder feature with AI / acceptance

✅ Expected result

cargo test reports 6 passed and 1 doctest; docs/design.md traces R1-R4 to named tests. Starter before you begin: Every test that calls an unfinished function fails with "not yet implemented" from todo!().

IF IT DOESN'T WORK

Test fails

Compare expected vs actual; check the boundary and the type, never edit the expected value.

Does not compile

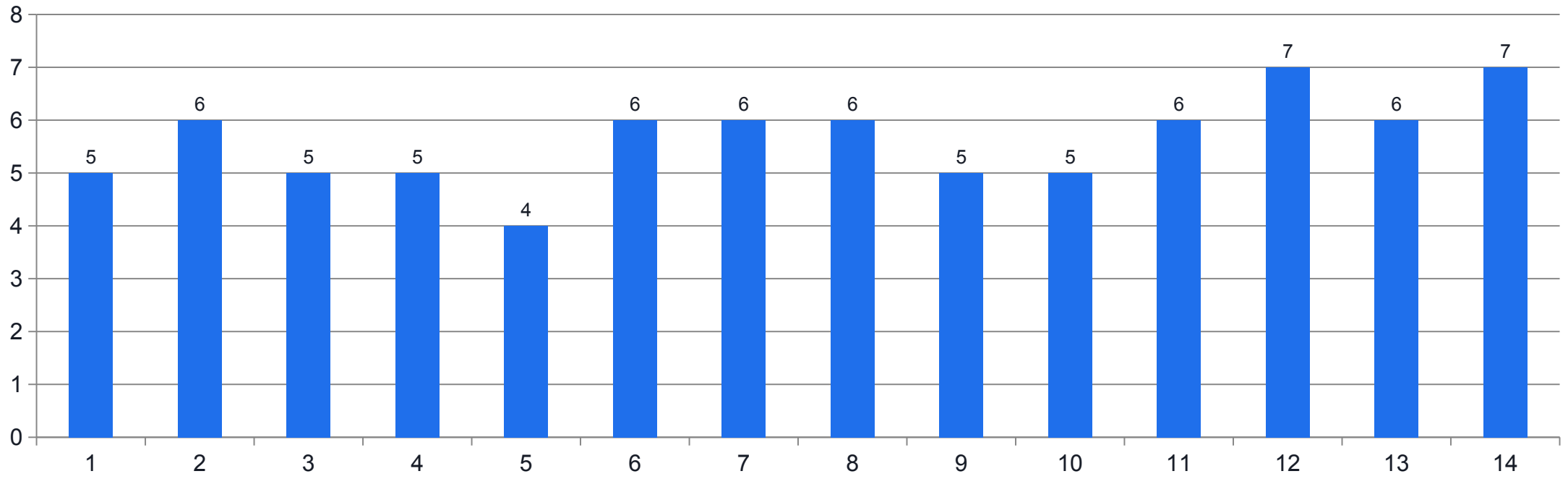
Read the first error and its code; rustc --explain E0XXX; ask the AI to explain before fixing.

clippy / fmt fails

Apply the suggested idiom or cargo fmt, then rerun cargo test.

© 2026 Tertiary Infotech Academy Pte Ltd

Every activity is verified: tests per reference crate



WHAT THE DATA SHOWS

All 14 solution crates pass cargo test (including doctests), clippy -D warnings and fmt --check. Every starter compiles and fails in the intended way until you complete it.

© 2026 Tertiary Infotech Academy Pte Ltd

SPECIFICATION TO EVIDENCE

What You Achieved

1

LO1

Determine basic software components using programming methodologies to meet functional specifications

2

LO2

Apply programming methodologies and tools for software creation

3

LO3

Select essential programming controls and features to meet software design requirements

4

LO4

Examine the interoperability and functionality of programming components

5

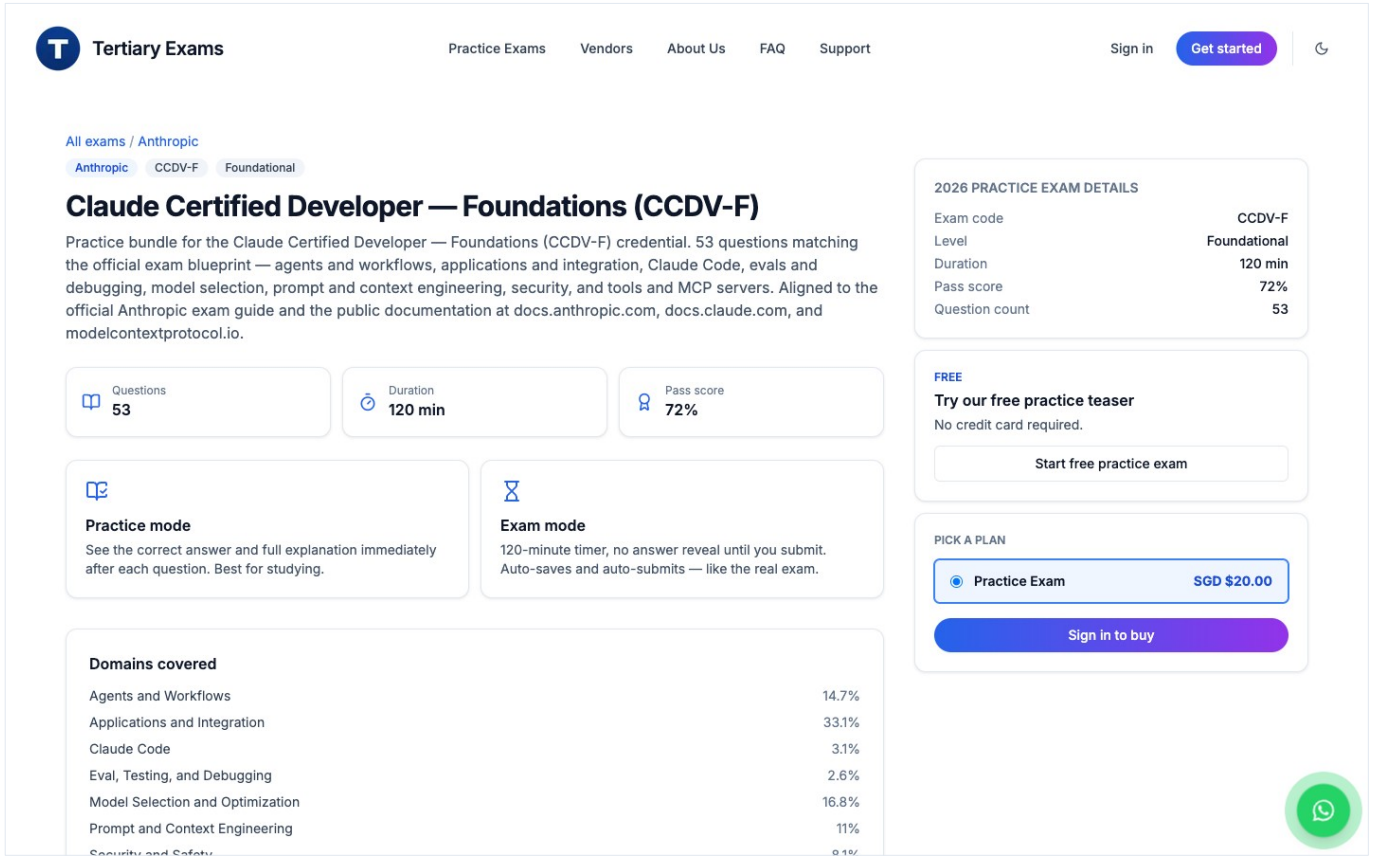
LO5

Generate programming design documentation aligned with user specifications

© 2026 Tertiary Infotech Academy Pte Ltd

PRACTICE EXAM

Practice Exam: keep practising AI-assisted development



Screenshot of exams.tertiaryinfotech.com (CCDV-F practice exam page), captured September 2026.

Where

exams.tertiaryinfotech.com - practice exams with exam and practice modes.

Recommended

Claude Certified Developer - Foundations (CCDV-F): AI coding tools, agents and debugging.

Free start

Try 10 questions free on any exam before deciding.

Not the WSQ test

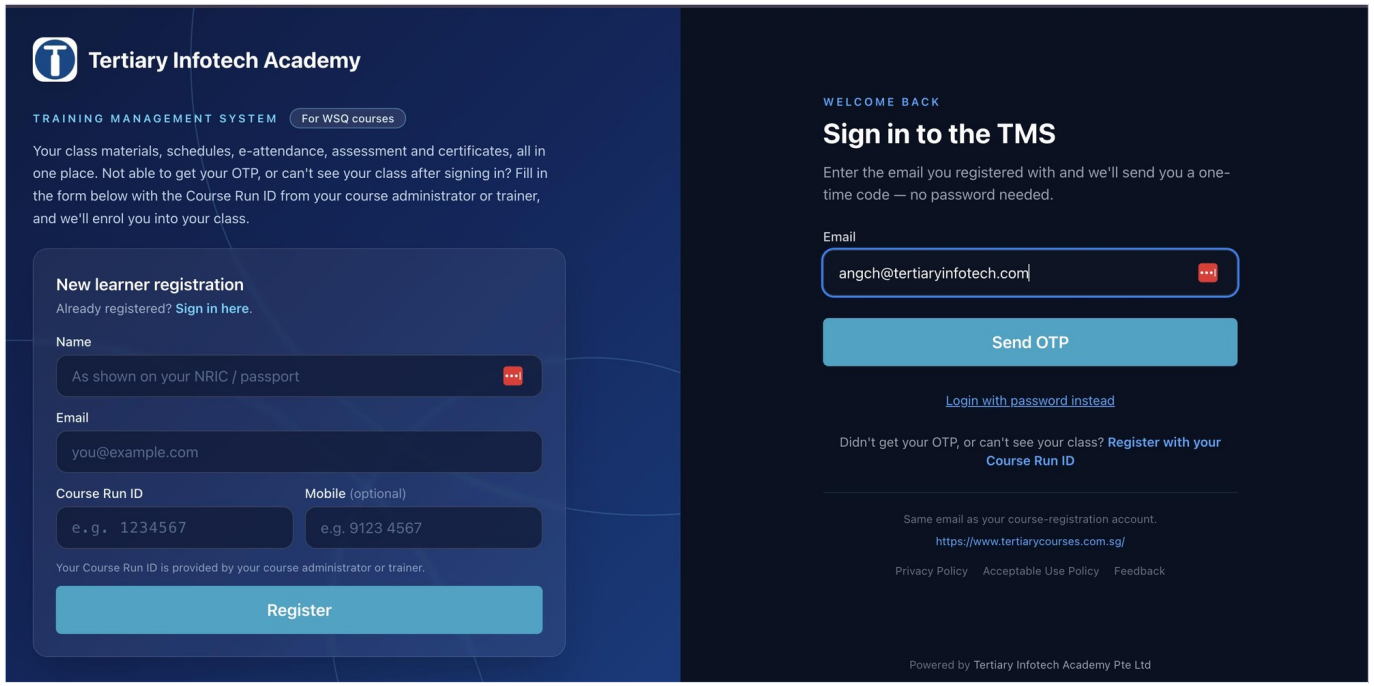
Optional further practice; it is not this course's assessment and there is no Rust-specific exam.

[Open the practice exam]

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNER ACCESS

Download course material from the LMS



Screenshot of the live LMS sign-in page (lms-tms.tertiaryinfotech.com), captured September 2026.

1. Sign in

Open lms-tms.tertiaryinfotech.com and sign in with your email (OTP) or password.

2. Open this course

AI Vibe Coding with Rust (TGS-2023039924).

3. Download

Learner slides PDF, Learner Guide and the Activities link.

4. Assessment

Question papers open on the assessment day; upload your answers there.

Open the LMS

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL DAY / 2 HOURS

Assessment / final reminder

Written Assessment

- Day 2, 4:30-5:30 PM
- 60 minutes / 5 questions
- K1-K5 / open book

Practical Performance

- Day 2, 5:30-6:30 PM
- 60 minutes / 5 tasks
- A1-A5 / Rust crates + evidence

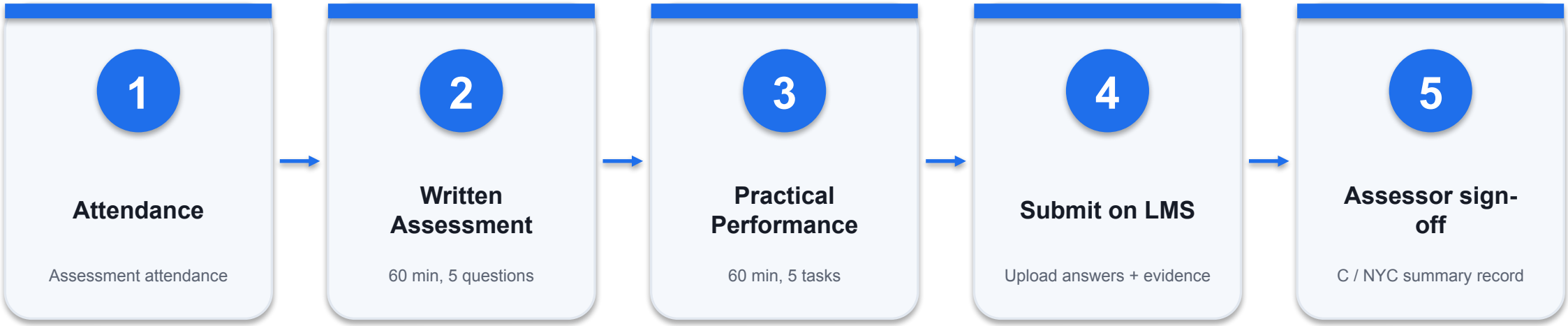
Result and funding

- All items attempted individually
- Competent (C) or NYC
- Funding requires 75% attendance + C

© 2026 Tertiary Infotech Academy Pte Ltd

OPEN BOOK / INDIVIDUAL

Assessment Flow



ORDER OF THE DAY

Written Assessment (K1-K5) 4:30-5:30 PM, then Practical Performance (A1-A5) 5:30-6:30 PM on Day 2. Follow the assessor's submission instructions.

© 2026 Tertiary Infotech Academy Pte Ltd

MANDATORY LEARNER ADMIN

Digital Attendance (Mandatory)

1

TRAQOM / SSG

Take the digital attendance through the Singpass app when the trainer displays the QR code.

2

AM, PM and assessment

Attendance is taken for each session and separately for the assessment.

3

75% minimum

At least 75% attendance is required for assessment eligibility and funding.

4

Your own record

Use only your own Singpass; tell the trainer if the course title or code does not match.

© 2026 Tertiary Infotech Academy Pte Ltd

TERTIARY INFOTECH ACADEMY

Thank You

Keep the loop: specify, prompt, compile, test, review, record.

enquiry@tertiaryinfotech.com / +65 6100 0613

© 2026 Tertiary Infotech Academy Pte Ltd